

EH整理的の板子（有一半不是自己写的）

这是题目对照。

EH整理的の板子（有一半不是自己写的）

A-基础算法

- A1 高精度加法
- A2 高精度减法
- A3 高精度乘法
- A4 高精度除法
- A5 二分查找
- A6 二分答案
- A7 分数规划
- A8 前缀和
 - 二维前缀和
- A10 差分
- A12 ST表
- A13 快速排序、第k小的数
- A14 归并排序、逆序对
- A15 堆
- A16 对顶堆
- A17 距离之和最小、中位数
- A18 双指针（定量）
- A19 双指针（定性）
- A20 双指针（异或释放）
- A21 双指针区间合并
- A22 堆序列合并
- A24 贪心
- A29 线段覆盖
- A41 三分小数
- A42 三分整数

B-搜索算法

- B6 DFS深搜
- B15 BFS广搜

C-数据结构

- C1 并查集
- C2 线段树
- C5 普通平衡树
- C8 可持续化线段树
- C19 KD树
- C111 莫队

D 图论

- D2 狄克斯特拉算法
- D4 最短路 Floyd 算法
- D5 全源 Johnson 算法
- D7、D8 Prim、Kruscal算法-最小生成树
- D9 倍增算法（LCA）
- D10 Tarjan算法（离线LCA）
- D11 树链剖分（最近公共祖先 LCA）
- D13 树上距离

E-动态规划

- E4 最长上升子序列（二分优化）
- E5 最长公共子序列
- E6 最长公共子串

- E8 01背包
- E9 完全背包
- E10 多重背包
- E11 滑动窗口
- E17 树形DP
- F-字符串
 - F3 KMP算法
 - F6 Trie字典树
 - F7 最大异或对
- G-数学
 - G1 快速幂
 - G2 高精度快速幂
 - G5 gcd及lcm问题
 - G3 矩阵快速幂
 - G8 线性筛质数
 - G9 筛法求欧拉函数
 - G10 筛法求因数个数
 - G11 筛法求约数和
 - G13 费马小定理
 - G14 拓展欧拉定理-超大幂次取余
 - G17 拓展欧几里得-不定方程
 - G18 拓展欧几里得-乘法逆元
 - G20 扩展中国剩余定理
 - G22 扩展BSGS算法
 - G23 高斯消元法
 - G27 求组合数-卢卡斯
 - G29 隔板法
 - G32 卡特兰数
 - G33 整除分块
 - G41 FFT-多项式乘法
 - G45+G46 第一、二类斯特林数
 - G49 向量运算
 - G50 线线关系
 - G52 凸包算法
 - G53 旋转卡壳
 - G99 超级gcd
- H 邪教
 - H1 哈希状态
 - H2 超绝浮点精度

A-基础算法

A1 高精度加法

原题描述：

高精度加法，相当于 $a+b$ problem，**不用考虑负数**。

输入格式：

分两行输入。 $a,b \leq 10^{500}$ 。

输出格式：

输出只有一行，代表 $a+b$ 的值。

输入输出样例：

输入 #1

```
1
1
```

输出 #1

```
2
```

输入 #2

```
1001
9099
```

输出 #2

```
10100
```

解析：时间复杂度 $O(n)$

A2 高精度减法

题目描述

高精度减法。

输入格式

两个整数 a, b （第二个可能比第一个大）。

输出格式

结果（是负数要输出负号）。

输入输出样例

输入 #1

```
2
1
```

输出 #1

```
1
```

解析：时间复杂度 $O(n)$

A3 高精度乘法

题目描述

高精度乘法模板题。

输入格式

两个非负整数 a, b ，求它们的乘积。

输出格式

输出一个非负整数表示乘积。每个非负整数不超过 10^{2000}

输入输出样例

输入 #1

```
2
1
```

输出 #1

```
2
```

解析：时间复杂度 $O(n^2)$

A4 高精度除法

题目描述

输入两个整数 a, b ，输出它们的商。

输入格式

两行，第一行是被除数 a ，第二行是除数 b 。

$0 \leq a \leq 10^{5000}$ ， $1 \leq b \leq 10^9$

输出格式

一行，商的整数部分。

输入输出样例

输入 #1

```
2
1
```

输出 #1

```
1
```

解析：时间复杂度 $O(n)$

A5 二分查找

题目描述

输入 n 个不超过 10^9 的单调不减的（就是后面的数字不小于前面的数字）非负整数 $a_1, a_2, \dots, a_n$ ，然后进行 m 次询问。对于每次询问，给出一个整数 q ，要求输出这个数字在序列中第一次出现的编号，如果没有找到的话输出 -1 。

输入格式

第一行 2 个整数 n 和 m ，表示数字个数和询问次数。

第二行 n 个整数，表示这些待查询的数字。

第三行 m 个整数，表示询问这些数字的编号，从 1 开始编号。

输出格式

输出一行， m 个整数，以空格隔开，表示答案。

输入输出样例

输入 #1

```
11 3
1 3 3 3 5 7 9 11 13 15 15
1 3 6
```

输出 #1

```
1 2 -1
```

数据保证， $1 \leq n \leq 10^6$ ， $0 \leq a_i, q \leq 10^9$ ， $1 \leq m \leq 10^5$

本题输入输出量较大，请使用较快的 IO 方式。

解析：时间复杂度 $O(m\log(n))$

对于py，使用bisect_left即可做到快速二分。

A6 二分答案

题目描述

木材厂有 n 根原木，现在想把这些木头切割成 k 段长度均为 l 的小段木头（木头有可能有剩余）。请求出 l 的最大值。

确保长度都是正整数。

例如有两根原木长度分别为 11 和 21，要求切割成等长的 6 段，很明显能切割出来的小段木头长度最长为 5。

输入格式

第一行是两个正整数 n, k ，分别表示原木的数量，需要得到的小段的数量。

接下来 n 行，每行一个正整数 L_i ，表示一根原木的长度。

输出格式

仅一行，即 l 的最大值。

如果连 1cm 长的小段都切不出来，输出 0。

输入输出样例

输入 #1

3 7
232
124
456

输出 #1

114

对于 100% 的数据，有 $1 \leq n \leq 10^5$, $1 \leq k \leq 10^8$, $1 \leq L_i \leq 10^8 (i \in [1, n])$ 。

解析：时间复杂度 $O(n \log(\max(L_i)))$

与传统的二分不同，这题不是 n 次询问，每项二分，而是直接二分答案，这与校省选的图书馆那题很像。

A7 分数规划

分数规划

用途：求分数的最值。

问题：每种物品有两个权值 a_i 和 b_i ,

选出若干个物品使得 $\frac{\sum a_i}{\sum b_i}$ 最大/最小。

等价：给出 a_i 和 b_i ，求一组 $w_i \in \{0,1\}$,

最大化或最小化 $\frac{\sum w_i \cdot a_i}{\sum w_i \cdot b_i}$ 。

若干个物品：满足给定 约束条件 的物品。

二分法

最大化：二分一个答案 x , $\frac{\sum w_i \cdot a_i}{\sum w_i \cdot b_i} \geq x$

$$\sum w_i \cdot a_i - x \sum w_i \cdot b_i \geq 0$$

$$\sum w_i \cdot (a_i - x b_i) \geq 0$$

如果 和式 ≥ 0 ，说明 x 是可行的，否则不可行。

最小化：二分一个答案 x , $\frac{\sum w_i \cdot a_i}{\sum w_i \cdot b_i} \leq x$

$$\sum w_i \cdot (a_i - x b_i) \leq 0$$

如果 和式 ≤ 0 ，说明 x 是可行的，否则不可行。

解题思路

1. 明确 a_i, b_i 是什么，把 $a_i - x b_i$ 作为新权值求和
2. 明确 约束条件 是什么，选择合适的算法求和

A8 前缀和

题目描述

给定 n 个正整数组成的数列 $a_1, a_2, \dots, a_n$ 和 m 个区间 $[l_i, r_i]$ ，分别求这 m 个区间的区间和。

对于所有测试数据， $n, m \leq 10^5, a_i \leq 10^4$

输入格式

第一行，为一个正整数 n 。

第二行，为 n 个正整数 $a_1, a_2, \dots, a_n$

第三行，为一个正整数 m 。

接下来 m 行，每行为两个正整数 l_i, r_i ，满足 $1 \leq l_i \leq r_i \leq n$

输出格式

共 m 行。

第 i 行为第 i 组答案的询问。

输入输出样例

输入 #1

```
4
4 3 2 1
2
1 4
2 3
```

输出 #1

```
10
5
```

对于 100% 的数据: $1 \leq n, m \leq 10^5$, $1 \leq a_i \leq 10^4$

解析: 时间复杂度 $O(n)$

把每项的和加起来, 然后利用相减的方式得出和的值。适用于静态的区间求和, 应用面非常广泛。

二维前缀和

题目描述

在一个 $n \times m$ 的只包含 0 和 1 的矩阵里找出一个不包含 0 的最大正方形, 输出边长。

输入格式

输入文件第一行为两个整数 n, m ($1 \leq n, m \leq 100$), 接下来 n 行, 每行 m 个数字, 用空格隔开, 0 或 1。

输出格式

一个整数, 最大正方形的边长。

输入输出样例

输入 #1

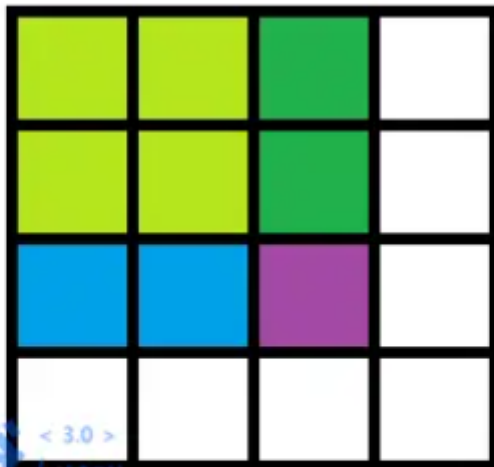
```
4 4
0 1 1 1
1 1 1 0
0 1 1 0
1 1 0 1
```

输出 #1

```
2
```

解析: EH不想写了





黄色是重叠部分，紫色是当前值，两个加上后减去重叠再加上当前值即为我们要求的。

A10 差分

题目描述

给定一个长度为 n 的数列 $a_1, a_2, \dots, a_n$ ，每次可以选择一个区间 $[l, r]$ ，使这个区间内的数都加 1 或者都减 1。

请问至少需要多少次操作才能使数列中的所有数都一样，并求出在保证最少次数的前提下，最终得到的数列有多少种。

输入格式

第一行一个正整数 n

接下来 n 行,每行一个整数,第 $i + 1$ 行的整数表示 a_i 。

输出格式

第一行输出最少操作次数

第二行输出最终能得到多少种结果

输入输出样例

输入 #1

```
4
1
1
2
2
```

输出 #1

```
1
2
```

对于 100% 的数据, $n \leq 100000, 0 \leq a_i \leq 2^{31}$ 。

解析：时间复杂度 $O(n)$

差分与前缀和相对，一个是区间求，一个是区间操作，好在都是静态的，不然就需要线段树了。

A12 ST表

给定一个长度为 N 的数列，和 M 次询问，求出每一次询问的区间内数字的最大值。

输入格式

第一行包含两个整数 N, M ，分别表示数列的长度和询问的个数。

第二行包含 N 个整数（记为 a_i ），依次表示数列的第 i 项。

接下来 M 行，每行包含两个整数 l_i, r_i ，表示查询的区间为 $[l_i, r_i]$ 。

输出格式

输出包含 M 行，每行一个整数，依次表示每一次询问的结果。

8 3	9
9 3 1 7 5 6 0 8	9
1 6	7
1 5	
2 7	

对于 100% 的数据，满足 $1 \leq N \leq 10^5$ ， $1 \leq M \leq 2 \times 10^6$ ， $a_i \in [0, 10^9]$ ， $1 \leq l_i \leq r_i \leq N$ 。

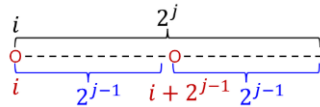
ST表 (Sparse Table, 稀疏表)

主要用来解决 RMQ（区间最大/最小值查询）问题。主要应用**倍增思想**，可以实现 $O(n \log n)$ 预处理、 $O(1)$ 查询。

1. 预处理ST表

倍增法递推：用两个等长小区间拼凑一个大区间

$f[i][j]$ 以第 i 个数起点，长度为 2^j 的区间中的最大值。



$$f[i][j] = \max(f[i][j-1], f[i + 2^{j-1}][j-1])$$

```
for(int i=1;i<=n;i++) scanf("%d",&f[i][0]);
for(int j=1;j<=20;j++) //枚举区间长度
    for(int i=1;i+(1<<j)-1<=n;i++) //枚举起点
        f[i][j]=max(f[i][j-1],f[i+(1<<(j-1))][j-1]);
```

区间终点： $i + 2^j - 1 \leq n$

$n = 6$

区间长度倍增：1,2,4,8,...

$f[i, 0]$: [1 1][2 2][3 3][4 4][5 5][6 6]

$f[i, 1]$: [1 2][2 3][3 4][4 5][5 6]

$f[i, 2]$: [1 4][2 5][3 6]

$j = 3$: $i + 2^j - 1 = 1 + 8 - 1 > 6$

ST表 (Sparse Table, 稀疏表)

主要用来解决RMQ (区间最大/最小值查询) 问题。主要应用倍增思想, 可以实现 $O(n \log n)$ 预处理、 $O(1)$ 查询。

2. 处理查询

对查询区间 $[l, r]$ 做分割、拼凑

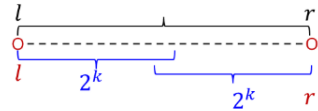
区间长度的指数 $k = \log_2(r - l + 1)$

```
k = 0: {1}
k = 1: {2, 3}
k = 2: {4, 5, 6, 7}
k = 3: {8, 9, 10, ..., 15}
```

观察发现: $2^k \leq r - l + 1 < 2 \cdot 2^k$,

即区间 $[l, r]$ 必可以用两个长度为 2^k 的区间重叠拼凑。

凡是符合结合律且可重复贡献的信息查询都可以使用ST表。显然最大值、最小值、最大公因数、最小公倍数、按位或、按位与 都符合这个条件。如果涉及区间修改操作, 就要使用线段树啦。



即 $\max(f[l][k], f[r - 2^k + 1][k])$

```
for(int i=1,l,r;i<=m;i++){
    scanf("%d%d",&l,&r);
    int k=log2(r-l+1); //区间长度的指数
    printf("%d\n",max(f[l][k],f[r-(1<<k)+1][k]));
}
```

$[1, 4] \rightarrow [1, 4] + [1, 4]$

$[1, 5] \rightarrow [1, 4] + [2, 5]$

$[1, 6] \rightarrow [1, 4] + [3, 6]$

$[1, 7] \rightarrow [1, 4] + [4, 7]$

A13 快速排序、第k小的数

输入 n ($1 \leq n < 5000000$ 且 n 为奇数) 个数字 a_i ($1 \leq a_i < 10^9$), 输出这些数字的第 k 小的数。

最小的数是第 0 小。

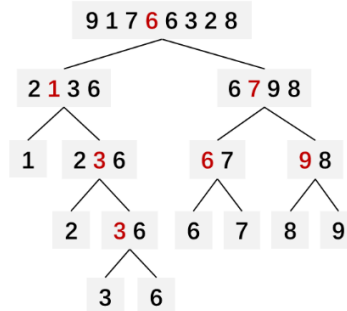
请尽量不要使用 `nth_element` 来写本题, 因为本题的重点在于练习分治算法。

```
5 1
4 3 2 1 5
```

2

```
int n,k,a[100010];

int qnth_element(int l, int r){
    if(l==r) return a[l];
    int i=l-1, j=r+1, x=a[(l+r)/2];
    while(i<j){
        do i++; while(a[i]<x); //向右找>=x的数
        do j--; while(a[j]>x); //向左找<=x的数
        if(i<j) swap(a[i],a[j]);
    }
    if(k<=j) return qnth_element(l,j);
    else return qnth_element(j+1,r);
}
```



每次取一半递归, i, j 指针每层走的步数为 $n, \frac{n}{2}, \frac{n}{4}, \dots, 1$,

总步数 $= n + \frac{n}{2} + \frac{n}{4} + \dots + 1 = 2n$,

所以时间复杂度为 $O(n)$ 。

A14 归并排序、逆序对

对于给定的一段正整数序列, 逆序对就是序列中 $a_i > a_j$ 且 $i < j$ 的有序对。注意序列中可能有重复数字。

输入格式

第一行, 一个数 n , 表示序列中有 n 个数。 $n \leq 5 \times 10^5$

第二行 n 个数, 表示给定的序列。序列中每个数字不超过 10^9

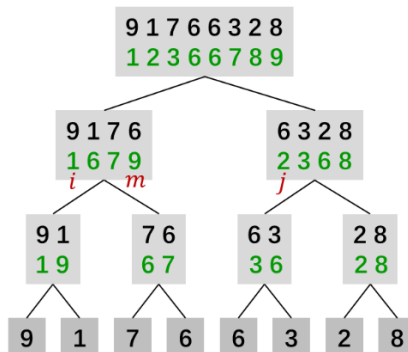
输出格式

输出序列中逆序对的数目。

```
4
5 2 6 1
```

4

每当从右段取数时, 统计逆序对的数目。



```
int n,a[500010],b[500010];
long long res;

void merge(int l,int r){
    if(l>=r) return;
    int mid=l+r>>1;
    merge(l,mid);
    merge(mid+1,r); //拆分

    int i=l,j=mid+1,k=1; //合并
    while(i<=mid && j<=r){
        if(a[i]<=a[j]) b[k++]=a[i++];
        else b[k++]=a[j++], res+=mid-i+1;
    }
    while(i<=mid) b[k++]=a[i++];
    while(j<=r) b[k++]=a[j++];
    for(i=l; i<=r; i++) a[i]=b[i];
}
```

A15 堆

给定一个数列，初始为空，请支持下面三种操作：

1. 给定一个整数 x ，请将 x 加入到数列中。
2. 输出数列中最小的数。
3. 删除数列中最小的数（如果有多个数最小，只删除 1 个）。

输入格式

第一行是一个整数，表示操作的次数 n 。

接下来 n 行，每行表示一次操作。每行首先有一个整数 op 表示操作类型。

若 $op=1$ ，则后面有一个整数 x ，表示要将 x 加入数列。

若 $op=2$ ，则表示要求输出数列中的最小数。

若 $op=3$ ，则表示删除数列中的最小数。如果有多个数最小，只删除 1 个。

输出格式

对于每个操作 2，输出一行一个整数表示答案。

```
5
1 2
1 5
2
3
2
```

```
2
5
```

对于 100% 的数据，保证 $1 \leq n \leq 10^6$ ， $1 \leq x < 2^{31}$

解析：好吧，EH不摆了。

py里的堆可以用heapq高效实现，你不需要知道它是什么，什么是堆（优先队列）。这样吧，你把一个py里的列表q想象成一个盒子，你每次花费 $\log n$ 的时间往里面丢东西（`heapq.heappush(q, ai)`），它总是会弹出盒子里最小那个数，查询方式为`q[0]`。你也可以花费 $\log n$ 的时间把这个最小数取出来（`heapq.heappop(q)`），此时其他数会成为最小的数并置顶。

heapq默认小根堆（弹出最小数），你可以通过加个负号让它变成大根堆。

A16 对顶堆

[Luogu P7072 \[CSP-J2020\] 直播获奖](#)

NOI2130 即将举行。为了增加观赏性，CCF 决定逐一评出每个选手的成绩，并直播即时的获奖分数线。

本次竞赛的获奖率为 $w\%$ ，即当前排名前 $w\%$ 的选手的最低成绩就是即时的分数线。

更具体地，若当前已评出了 p 个选手的成绩，则当前计划获奖人数为 $\max(1, \lfloor p \times w\% \rfloor)$ ，其中 w 是获奖百分比， $\lfloor x \rfloor$ 表示对 x 向下取整， $\max(x, y)$ 表示 x 和 y 中较大的数。

如有选手成绩相同，则所有成绩并列的选手都能获奖，因此实际获奖人数可能比计划中多。

输入格式

第 1 行两个正整数 n, w 。

分别代表选手总数与获奖率。（ $n \leq 10^5$ ）

第 2 行有 n 个非负整数，

依次代表逐一评出的选手成绩。

输出格式

只有一行，包含 n 个非负整数，

依次代表选手成绩逐一评出后，

即时的获奖分数线。

```
10 60
200 300 400 500 600 600 0 300 200 100
```

```
200 300 400 400 400 500 400 400 300 300
```

已评测选手人数	1	2	3	4	5	6	7	8	9	10
计划获奖人数	1	1	1	2	3	3	4	4	5	6
已评测选手的分数从高到低排列（其中，分数线用 粗体 标出）	200	300 200	400 300 200	500 400 300 200	600 500 400 300 200	600 600 500 400 300 200	600 600 500 400 300 200 0	600 600 500 400 300 200 0	600 600 500 400 300 200 0	600 600 500 400 300 300 200 100 0

注意，在第 9 名选手的成绩评出之后，计划获奖人数为 5 人，但由于有并列，因此实际会有 6 人获奖。

洛谷

解析：

对顶堆

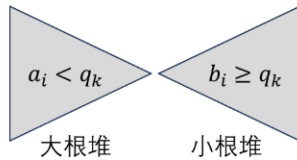
可以动态维护一个序列上第 k 大的数， k 值会发生变化。比写线段树或 BST 简单。

对顶堆由一个大根堆与一个小根堆组成，小根堆维护前 k 大的数（包含第 k 个），大根堆维护比第 k 大数小的数。

1. **插入**：若插入的元素 $\geq$ 小根堆堆顶元素，则将其插入小根堆，否则将其插入大根堆。
2. **维护**：当小根堆的大小 $> k$ 时，不断将小根堆堆顶元素取出并插入大根堆，直到小根堆的大小等于 k ；
当小根堆的大小 $< k$ 时，不断将大根堆堆顶元素取出并插入小根堆，直到小根堆的大小等于 k 。

3. **查询第 k 大元素**：小根堆堆顶元素。

4. **删除第 k 大元素**：删除小根堆堆顶元素。



```
priority_queue<int> a; //大根堆
priority_queue<int,vector<int>,greater<int>> b;

for(int i=1; i<=n; i++){
    int x; scanf("%d",&x);
    if(b.empty()||x>=b.top()) b.push(x); //插入
    else a.push(x);

    while(b.size()>k) a.push(b.top()), b.pop(); //调整
    while(b.size()<k) b.push(a.top()), a.pop();
    printf("%d ", b.top()); b.pop(); //取值，删除
}
```

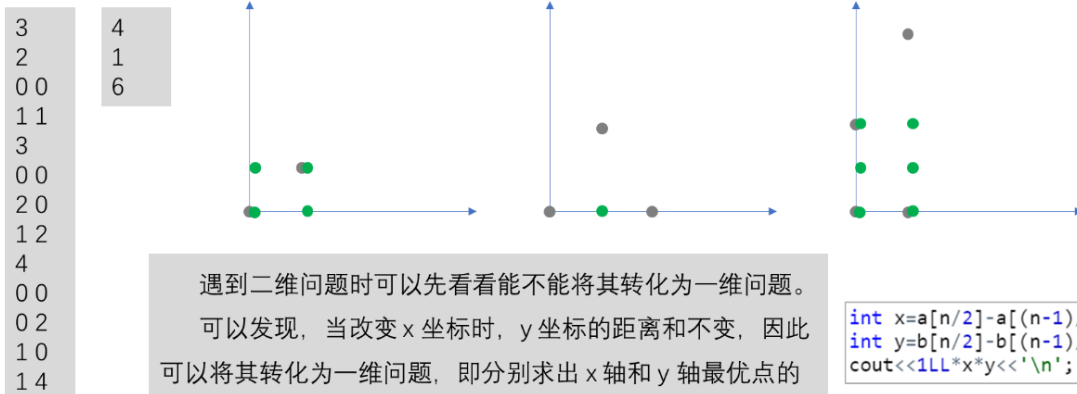
插入、删除后，小根堆的大小与期望的 k 值最多相差 1，故每次维护最多只需对大根堆与小根堆中的元素进行一次调整，这些操作的时间复杂度都是 $O(\log n)$

A17 距离之和最小、中位数

题意翻译

二维平面上有 n 个点，要找一个点，使得所有点到它的曼哈顿距离（ x 和 y 的坐标差距之和）之和最小。

请问有几个满足该要求的点？ $1 \leq t \leq 1000$, $1 \leq n \leq 1000$, $0 \leq x_i, y_i \leq 10^9$ ，坐标均为整数。



遇到二维问题时可以先看看能不能将其转化为一维问题。
可以发现，当改变 x 坐标时， y 坐标的距离和不变，因此
可以将其转化为一维问题，即分别求出 x 轴和 y 轴最优点的
数量，然后将其相乘就是答案。

```
int x=a[n/2]-a[(n-1)/2]+1;
int y=b[n/2]-b[(n-1)/2]+1;
cout<<1LL*x*y<<"\n";
```

A18 双指针（定量）

对一个给定的正整数 M ，求出所有的连续的正整数段（每一段至少有两个数），这些连续的自然数段中的全部数之和为 M 。

例子：1998 + 1999 + 2000 + 2001 + 2002 = 10000，所以从 1998 到 2002 的一个自然数段为 $M = 10000$ 的一个解。

输入格式

包含一个整数的单独一行给出 M 的值 ($10 \leq M \leq 2,000,000$)。

输出格式



```
int i=1,j=1,sum=1;
while(i<=m/2){ //i<=j
    if(sum<m){
        j++;
        sum+=j;
    }
    if(sum>=m){
        if(sum==m) printf("%d %d\n",i,j);
        sum-=i;
        i++;
    }
}
```

每行两个正整数，给出一个满足条件的连续正整数段中的第一个数和最后一个数，两数之间用一个空格隔开，所有输出行的第一个按从小到大的升序排列，对于给定的输入数据，保证至少有一个解。

把自然数看做一个升序序列， i 指针指向一段数的左端， j 指针指向这段数的右端。 sum 维护这段数的和。如果 $sum < M$ ，则 $j++$, $sum += j$ ；如果 $sum \geq M$ ，则 $sum -= i$, $i++$ 。注意 $sum == M$ 时，输出答案。

A19 双指针（定性）

灵梦有 n 个单词想要背，但她想通过一篇文章中的一段来记住这些单词。

文章由 m 个单词构成，她想在文章中找到连续的一段，其中包含最多的她想要背的单词（重复的只算一个）。并且在背诵的单词量尽量多的情况下，还要使选出的文章段落尽量短，这样她就可以用尽量短的时间学习尽可能多的单词了。

输入格式

第 1 行一个数 n ，接下来 n 行每行是一个长度不超过 10 的字符串，表示一个要背的单词。

接着是一个数 m ，然后是 m 行长度不超过 10 的字符串，每个表示文章中的一个单词。

输出格式

输出文件共 2 行。第 1 行为文章中最多包含的要背的单词数，第 2 行表示在文章中包含最多要背单词的最短连续段的长度。

$n \leq 1000$, $m \leq 10^5$ 。

3
hot dog milk
6
hot car dog milk hot car

3
3

要找的区间具有伸缩的特性，用双指针控制区间长度的变化，优化答案即可。

s[] 记录文章中的单词
word[] 记录单词表中的单词
cnt[] 记录文章当前区间内目标单词的出现次数
sum 记录文章中出现的目标单词个数
len 记录包含目标单词最多的区间的最短长度

i 指针指向当前区间的左端, j 指针指向当前区间的右端。

1. 如果新来的单词 s[j] 是目标单词, cnt[s[j]]++;

2. if(cnt[s[j]]==1) sum++, len=j-i+1;

3. while(i<=j):

if(cnt[s[i]]==1) break; //保持 i 指针位置不动

if(cnt[s[i]]>=2) cnt[s[i]]--, i++; //去重, 更优

if(!word[s[i]]) i++; //去掉非目标单词, 更优

4. 更新最短长度 len;

3: hot dog milk
6: hot car dog milk hot car
 i j

A20 双指针 (异或释放)

给你一串数 a

求出满足 $a_l + \dots + a_r = a_l \text{ xor } \dots \text{ xor } a_r$ ($l \leq r$) 的方案数。

$1 \leq n \leq 200000$, $0 \leq a_i < 2^{20}$ (1048576)

5
1 2 4 3 1

9

1 = 1
1 + 2 = 1 xor 2
1 + 2 + 4 = 1 xor 2 xor 4
2 = 2
2 + 4 = 2 xor 4
4 = 4
4 + 3 = 4 xor 3
3 = 3
1 = 1

1: 001

2: 010

4: 100

3: 011

1: 001

- 异或和是不进位的加法, 有 $a \oplus b \leq a + b$, 当二进制的同位数字不同时为 1 时, 才取等号。
- 搜索区间具有伸缩的特性, 用双指针维护。

1. i 指针指向 1, j 指针指向 0。
2. j 指针做预判, 成功则 $j++$, $s1+=a[j]$, $s2^+=a[j]$;
循环直到预判失败, 退出, j 指向合法区间右端;
3. $ans+=j-i+1$;
4. $s1-=a[i]$, $s2^-=a[i]$, $i++$;

```

5
1 2 4 3 1
i   j

```

```

1 = 1
1 + 2 = 1 xor 2
1 + 2 + 4 = 1 xor 2 xor 4

```

A21 双指针区间合并

[Luogu P1496 火烧赤壁](#)

连环线上的船舰多处起火。

曹操气急败坏的把你找来, 要你钻入火海把连环线上着火的船只的长度统计出来!

给定每个起火部分的起点和终点, 请你求出燃烧位置的长度之和。

输入格式

第一行一个整数, 表示起火的信息条数 n 。

接下来 n 行, 每行两个整数 a, b , 表示一个着火位置的起点和终点 (注意: 左闭右开)。

输出格式

输出一行一个整数表示答案。

```

3
-1 1
5 11
2 9

```

```
11
```

保证 $1 \leq n \leq 2 \times 10^4$, $-2^{31} \leq a \leq b < 2^{31}$, 且答案小于 2^{31} 。

对区间按起点排序,
扫描一遍, 合并区间, 计算区间长度。

A22 堆序列合并

Luogu P1631 序列合并

有两个长度为 N 的单调不降序列 A, B ，在 A, B 中各取一个数相加可以得到 N^2 个和，求这 N^2 个和中最小的 N 个。

输入格式

第一行一个正整数 N ；

第二行 N 个整数 $A_1 \dots A_N$ 。

第三行 N 个整数 $B_1 \dots B_N$ 。

4	3 5 6 7
2 6 6 7	
1 3 4 6	

让 A 中的每个数与 B 中一个数相加，生成 N 个有序队列，

1: $A[1] + B[1] \leq A[2] + B[1] \leq \dots \leq A[N] + B[1]$

2: $A[1] + B[2] \leq A[2] + B[2] \leq \dots \leq A[N] + B[2]$

k: $\dots$

N: $A[1] + B[N] \leq A[2] + B[N] \leq \dots \leq A[N] + B[N]$

输出格式

一行 N 个整数，从小到大表示这 N 个最小的和。

$1 \leq N \leq 10^5$, $1 \leq a_i, b_i \leq 10^9$ 。

要将这 N 个有序队列进行合并排序：

1. 将这 N 个队列中的第一列元素放入一个小根堆中；

2. 每次取出堆中的最小值。若这个最小值来自于第 k 个队列，就将第 k 个队列的下一个元素放入堆中。

时间复杂度： $O(N \log N)$ 。

A24 贪心

Luogu P1842 [USACO05NOV] 奶牛玩杂技

每头牛都有自己的体重以及力量，编号为 i 的奶牛的体重为 W_i ，力量为 S_i 。

当某头牛身上站着另一些牛时它就会在一定程度上被压扁，我们不妨把它被压扁的程度叫做它的压扁指数。

对于任意的牛，她的压扁指数等于摺在她上面的所有奶牛的总重（当然不包括她自己）减去它的力量。

奶牛们按照一定的顺序摺在一起后，她们的总压扁指数就是被压得最扁的那头奶牛的压扁指数。

你的任务就是帮助奶牛们找出一个摺在一起的顺序，使得总压扁指数最小。

输入

第一行一个整数 N 。

接下来 N 行，每行两个整数 W_i, S_i 。

4	5	4 3	-3
1 10		2 6	-2
2 6		8 1	5
4 3		1 10	4
8 1			

输出

一行一个整数表示最小总压扁指数。

直觉猜想：越重越壮的牛放在下面，被压扁指数越小？

如果正确，不妨按体重与力量之和排序，枚举计算答案。

对于 100% 的数据， $1 \leq N \leq 5 \times 10^4$, $1 \leq W_i \leq 10^4$, $1 \leq S_i \leq 10^9$ 。

A29 线段覆盖

现在各大 oj 上有 n 个比赛，每个比赛的开始、结束的时间点是知道的。

yyy 认为，参加越多的比赛，noip 就能考的越好（假的）。

所以，他想知道他最多能参加几个比赛。

由于 yyy 是蒟蒻，如果要参加一个比赛必须善始善终，而且不能同时参加 2 个及以上的比赛。

输入格式

第一行是一个整数 n ，接下来 n 行每行是 2 个整数 a_i, b_i ($a_i < b_i$)，表示比赛开始、结束的时间。

输出格式

一个整数最多参加的比赛数目。

3	2
0 2	
2 4	
1 3	

n 个区间通常看做 n 条线段。

目标就是找出不重叠的线段的最大条数。

贪心思想：结束越早的时段，越应该优先选择。

先对所有线段按右端点排序，然后枚举判断。

如果当前线段的左端点 $\geq$ 上一条线段的右端点，则可选。

A41 三分小数

题目描述

给定 n 个二次函数 $f_1(x), f_2(x), \dots, f_n(x)$ (均形如 $ax^2 + bx + c$)，设 $F(x) = \max\{f_1(x), f_2(x), \dots, f_n(x)\}$ ，求 $F(x)$ 在区间 $[0, 1000]$ 上的最小值。

输入格式

输入第一行为正整数 T ，表示有 T 组数据。

每组数据第一行一个正整数 n ，接着 n 行，每行 3 个整数 a, b, c ，用来表示每个二次函数的 3 个系数，注意二次函数有可能退化成一个一次函数。

输出格式

每组数据输出一行，表示 $F(x)$ 在区间 $[0, 1000]$ 上的最小值。答案精确到小数点后四位，四舍五入。

输入 #1

```
2
1
2 0 0
2
2 0 0
2 -4 2
```

输出 #1

```
0.0000
0.5000
```

对于 100% 的数据， $T < 10$ ， $n \leq 10^4$ ， $0 \leq a \leq 100$ ， $|b| \leq 5 \times 10^3$ ， $|c| \leq 5 \times 10^3$ 。

A42 三分整数

描述

定义含绝对值的一元一次函数 $f(x) = |kx + a| + b (x \in \mathbb{Z})$, 其中 k 为由外部给定的正整数, a, b 为整数。

现在, 对于给定的 n 个含绝对值的一元一次函数, 定义 $F(x) = f_1(x) + f_2(x) + \cdots + f_n(x)$, 求解 $F(x)$ 在区间 $[l, r]$ 上的最小值。

输入描述:

每个测试文件均包含多组测试数据。第一行输入一个整数 $T (1 \leq T \leq 5 \times 10^5)$ 代表数据组数, 每组测试数据描述如下:

第一行输入三个整数 $n, l, r (1 \leq n \leq 5 \times 10^5; -10^7 \leq l \leq r \leq 10^7)$ 代表 $f(x)$ 的数量、值域。

此后 n 行, 第 i 行输入三个整数 $k_i, a_i, b_i (0 \leq k_i, |a_i|, |b_i| \leq 10^6)$ 用于描述第 i 个含绝对值的一元一次函数。

除此之外, 保证所有的 n 之和不超过 5×10^5 。

输出描述:

对于每一组测试数据, 在一行上输出一个整数, 代表 $F(x)$ 在值域上的最小值。

示例1

输入: 3

4 -15 20
1 -11 2
2 -2 0
1 20 -20
0 0 -10
3 10 100000
1 -11 -4
2 5 -10
1 -13 -10
3 12 100000
1 -11 -4
2 5 -10
1 -13 -10

输出: 3
5
7

复制

复制

B-搜索算法

B6 DFS深搜

Luogu P1219 [USACO1.5]八皇后

一个如下的 6×6 的棋盘, 有六个棋子被放置在棋盘上, 使得每行、每列有且只有一个, 每条对角线 (包括两条主对角线的所有平行线) 上至多有一个棋子。

图中的布局可以用序列 2 4 6 1 3 5 来描述, 第 i 个数字表示在第 i 行的相应位置有一个棋子。

这只是棋子放置的一个解。请编一个程序找出所有棋子放置的解。

并把它们上面的序列方法输出, 解按字典顺序排列。请输出前 3 个解。最后一行是总方案数。

输入格式

一行一个正整数 n , 表示棋盘是 $n \times n$ 大小的。 $6 \leq n \leq 13$ 。

输出格式

前三行为前三个解, 每个解的两个数字之间用一个空格隔开。第四行只有一个数字, 表示解的总数。

输入输出样例

```
6      2 4 6 1 3 5
      3 6 2 5 1 4
      4 1 5 2 6 3
      4
```

0	1	2	3	4	5	6
1		○				
2				○		
3						○
4	○					
5			○			
6					○	

➤ DFS 搜索方案

1. 从第1行开始放, 然后尝试放第 $2 \cdots n$ 行。
2. 对于第 i 行, 依次枚举第 $1 \cdots n$ 列, 如果第 j 列能放下, 则记住位置, 宣布占领该位置 (i, j) 的辐射区域, 然后继续搜索第 $i+1$ 行。
3. 如果第 $i+1$ 行的 n 列均放不下, 则退回第 i 行的状态空间, 恢复现场, 尝试第 i 行的下一列。
4. 如果能放满 n 行, 说明找到了一种合法方案, 则 $ans+1$, 打印方案, 接着返回上一行, 继续搜索另一个合法方案, 直到搜完所有可能方案。
5. 因为是逐行逐列搜的, 先搜到的字典序一定最小。

B15 BFS广搜

Luogu P1588 [USACO07OPEN]Catch That Cow S

FJ丢失了他的一头牛, 他决定追回他的牛。已知FJ和牛在一条直线上, 初始位置分别为 x 和 y , 假定牛在原地不动。FJ的行走方式很特别: 他每一次可以前进一步、后退一步或者直接走到 $2 \times x$ 的位置。计算他至少需要几步追上他的牛。

输入格式

第一行为一个整数 $t(\leq 10)$, 表示数据组数; 接下来每行包含一个两个正整数 x 和 $y(0 < x, y \leq 10^5)$, 分别表示FJ和牛的坐标。

输出格式

对于每组数据, 输出最少步数。

输入输出样例

输入 #1

```
1
5 17
```

1. 暴力搜索当前位置的三种移动方式,
2. 最小步数应该用 BFS,
3. 从起始位置 x 开始搜索, 开个 dis 数组存储移动步数, 当 $x=y$ 时, $dis[y]$ 即答案

```
5-1=4
4*2=8
8*2=16
16+1=17
```

复制

4

C-数据结构

C1 并查集

题目描述

如题，现在有一个并查集，你需要完成合并和查询操作。

输入格式

第一行包含两个整数 N, M ,表示共有 N 个元素和 M 个操作。

接下来 M 行，每行包含三个整数 Z_i, X_i, Y_i 。

当 $Z_i = 1$ 时，将 X_i 与 Y_i 所在的集合合并。

当 $Z_i = 2$ 时，输出 X_i 与 Y_i 是否在同一集合内，是的输出

Y；否则输出 N。

输出格式

对于每一个 $Z_i = 2$ 的操作，都有一行输出，每行包含一个大写字母，为 Y 或者 N。

输入 #1

```
4 7
2 1 2
1 1 2
2 1 2
1 3 4
2 1 4
1 2 3
2 1 4
```

输出 #1

```
N
Y
N
Y
```

对于 100% 的数据， $1 \leq N \leq 2 \times 10^5$ ， $1 \leq M \leq 10^6$ ， $1 \leq X_i, Y_i \leq N$ ， $Z_i \in \{1, 2\}$ 。

解析：

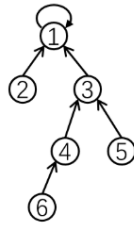
这个应用非常广泛，上述题目的正解的时间复杂度为 $O(n)$ ，以下为解释：

并查集是一种树形的数据结构。

支持两种操作：

合并：将两个子集合并成一个集合。

查找：确定某个元素处在哪个集合。



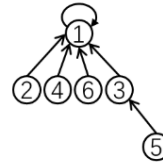
$fa[x]$ 存节点 x 的父节点。

例如， $fa[1]=1, fa[2]=1, fa[3]=1, fa[4]=3, fa[6]=4$ 。

$fa[]$ 初始化：

每个节点是一个集合，每个节点的父节点是它自己。

`for(int i=1; i<=n; i++) fa[i]=i;`



查找

根节点是集合的代表，查找就是找到元素所在集合的根。

1. 如果父节点等于自己，则找到了根并返回。
2. 如果还没找到根，则继续递归查找。

```
int find(int x){
    if(fa[x]==x) return x;
    return find(fa[x]);
}
```

例 $find(6)$

进入6
进入4
进入3
进入1, return 1
回到3, return 1
回到4, return 1
回到6, return 1

带路径压缩的查找

在返回的路上，顺带修改各节点的父节点为根。

```
int find(int x){
    if(fa[x]==x) return x;
    return fa[x]=find(fa[x]);
}
```

例 $find(6)$

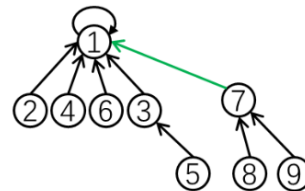
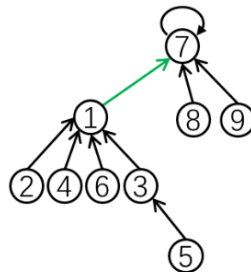
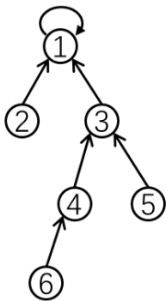
进入6
进入4
进入3
进入1, return 1
回到3, $fa[3]=1$, return 1
回到4, $fa[4]=1$, return 1
回到6, $fa[6]=1$, return 1

合并

把一个集合的根指向另一个集合的根。

```
void unionset(int x, int y){
    fa[find(x)]=find(y);
}
```

例 $unionset(6, 9)$



按秩合并（启发式合并）

把小集合的根指向大集合的根。

//记录并初始化子树的大小为1

`vector<int> siz(N, 1);`

`void unionset(int x, int y){`

`x=find(x), y=find(y);`

`if(x==y) return;`

`if(siz[x]>siz[y]) swap(x, y);`

`fa[x]=y;`

`siz[y]+=siz[x];`

`}`

- **fa[x]** 存节点 x 的父节点。
- **fa 初始化**：每个节点指向它自己。
- **查找**：**路径压缩**，使树扁平化。
- **合并**：把一个集合的根指向另一个集合的根。
- **按秩合并**：把小集合的根指向大集合的根。
- **时间复杂度**： $O(1)$

C2 线段树

题目描述

如题，已知一个数列 $\{a_i\}$ ，你需要进行下面两种操作：

1. 将某区间每一个数加上 k 。
2. 求出某区间每一个数的和。

输入格式

第一行包含两个整数 n, m ，分别表示该数列数字的个数和操作的总个数。

第二行包含 n 个用空格分隔的整数 a_i ，其中第 i 个数字表示数列第 i 项的初始值。

接下来 m 行每行包含 3 或 4 个整数，表示一个操作，具体如下：

1. **1 x y k**：将区间 $[x, y]$ 内每个数加上 k 。
2. **2 x y**：输出区间 $[x, y]$ 内每个数的和。

输出格式

输出包含若干行整数，即为所有操作 2 的结果。

输入 #1

```
5 5
1 5 4 2 3
2 2 4
1 2 3 2
2 3 4
1 1 5 1
2 1 4
```

输出 #1

```
11
8
20
```

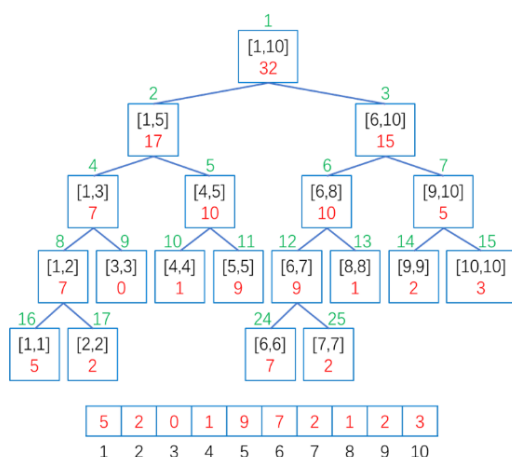
对于 100% 的数据: $1 \leq n, m \leq 10^5$, a_i, k 为正数, 且任意时刻数列的和不超过 2×10^{18} 。

【样例解释】

操作次数	输入内容	操作	数列					输出结果
			1	2	3	4	5	
0			1	5	4	2	3	
1	2 2 4	求出 [2, 4] 所有数的和	1	5	4	2	3	11
2	1 2 3 2	将 [2, 3] 内所有数加2	1	7	6	2	3	
3	2 3 4	求出 [3, 4] 所有数的和	1	7	6	2	3	8
4	1 1 5 1	将 [1, 5] 内所有数加1	2	8	7	3	4	
5	2 1 4	求出 [1, 4] 所有数的和	2	8	7	3	4	20

解析:

线段树是基于分治思想的二叉树, 用来维护区间信息 (区间和, 区间最值, 区间GCD等), 可以在 $\log n$ 的时间内执行**区间修改**和**区间查询**。
 线段树中每个**叶子节点**存储元素本身, **非叶子节点**存储区间内元素的统计值。



入1 {1,10,}
 入2 {1,5,}
 入4 {1,3,}
 入8 {1,2,}
 入16 {1,1,5}
 离16
 回8
 入17 {2,2,2}
 离17
 回4
 入9 {3,3,0}
 离9
 离4 .sum=7

 离1 .sum=32

1. 节点数组 $tr[]$

结构体包含三个变量: l, r, sum ,
 l, r 存区间的左右端点, sum 存区间和。

2. 递归建树

父节点编号为 p ,
 左孩子编号为 $2*p$, 右孩子编号为 $2*p+1$ 。

```
#define lc p<<1
#define rc p<<1|1
#define N 500005
int n,w[N];
struct node{
    int l,r,sum;
}tr[N*4];

void build(int p,int l,int r){
    tr[p]={l,r,w[l]};
    if(l==r) return; //是叶子则返回
    int m=l+r>>1; //不是叶子则裂开
    build(lc,l,m);
    build(rc,m+1,r);
    tr[p].sum=tr[lc].sum+tr[rc].sum;
}
```

4. 区间查询

拆分与拼凑的思想。例如, 查询区间 $[4,9]$ 可以拆分成 $[4,5]$, $[6,8]$ 和 $[9,9]$, 通过合并这三个区间的答案求得查询答案。

从根节点进入, **递归**执行以下过程:

1. 若查询区间 $[x,y]$ **完全覆盖**当前节点区间, 则立即回溯, 并返回该节点的 sum 值。
2. 若左子节点与 $[x,y]$ 有重叠, 则递归访问左子树。
3. 若右子节点与 $[x,y]$ 有重叠, 则递归访问右子树。

5. 区间修改

例如，对区间 $[4,9]$ 内的每个数加上 5。如果修改区间 $[x,y]$ 所覆盖的每个叶子节点，时间将是 $O(n)$ 的。

我们做**懒惰修改**，当 $[x,y]$ **完全覆盖** 节点区间 $[a,b]$ 时，先修改该区间的 sum 值，再打上**一个“懒标记”**，然后立即返回。等下次需要时，再下传“懒标记”。这样，可以把每次修改和查询的时间都控制到 $O(\log n)$

老子的记账本

C5 普通平衡树

题目描述

您需要动态地维护一个可重集合 M ，并且提供以下操作：

1. 向 M 中插入一个数 x 。
2. 从 M 中删除一个数 x （若有多个相同的数，应只删除一个）。
3. 查询 M 中有多少个数比 x 小，并且将得到的答案加一。
4. 查询如果将 M 从小到大排列后，排名位于第 x 位的数。
5. 查询 M 中 x 的前驱（前驱定义为小于 x ，且最大的数）。
6. 查询 M 中 x 的后继（后继定义为大于 x ，且最小的数）。

对于操作 3,5,6，**不保证**当前可重集中存在数 x 。

输入格式

第一行为 n ，表示操作的个数，下面 n 行每行有两个数 opt 和 x ， opt 表示操作的序号（ $1 \leq opt \leq 6$ ）

输出格式

对于操作 3, 4, 5, 6 每行输出一个数，表示对应答案。

输入 #1

```
10
1 106465
4 1
1 317721
1 460929
1 644985
1 84185
1 89851
6 81968
1 492737
5 493598
```

输出 #1

```
106465
84185
492737
```

【数据范围】

对于 100% 的数据, $1 \leq n \leq 10^5$, $|x| \leq 10^7$

C8 可持续化线段树

题目背景

这是个非常经典的可持续化权值线段树入门题——静态区间第 k 小。

题目描述

如题, 给定 n 个整数构成的序列 a , 将对于指定的闭区间 $[l, r]$ 查询其区间内的第 k 小值。

输入格式

第一行包含两个整数, 分别表示序列的长度 n 和查询的个数 m 。

第二行包含 n 个整数, 第 i 个整数表示序列的第 i 个元素 a_i 。

接下来 m 行每行包含三个整数 l, r, k , 表示查询区间 $[l, r]$ 内的第 k 小值。

输出格式

对于每次询问, 输出一行一个整数表示答案。

输入 #1

```
5 5
25957 6405 15770 26287 26465
2 2 1
3 4 1
4 5 1
1 2 2
4 4 1
```

输出 #1

```
6405
15770
26287
25957
26287
```

- 对于 100% 的数据, 满足 $1 \leq n, m \leq 2 \times 10^5$, $0 \leq a_i \leq 10^9$, $1 \leq l \leq r \leq n$, $1 \leq k \leq r - l + 1$ 。

解析: 时间复杂度: $O(n \log n)$, $\log n$ 查询, 原理不解释, 毕竟EH看不懂。

C19 KD树

题目描述

给定平面上 n 个点, 找出其中的一对点的距离, 使得在这 n 个点的所有点对中, 该距离为所有点对中最小的

输入格式

第一行: n , 保证 $2 \leq n \leq 200000$ 。

接下来 n 行: 每行两个实数: $x\ y$, 表示一个点的行坐标和列坐标, 中间用一个空格隔开。

输出格式

仅一行, 一个实数, 表示最短距离, 精确到小数点后面 4 位。

输入输出样例 #1

输入 #1

```
3
1 1
1 2
2 2
```

输出 #1

```
1.0000
```

数据保证 $0 \leq x, y \leq 10^9$

C111 莫队

题目描述

小B 有一个长为 n 的整数序列 a , 值域为 $[1, k]$ 。 他一共有 m 个询问, 每个询问给定一个区间 $[l, r]$,

求: $\sum_{i=1}^k c_i^2$

其中 c_i 表示数字 i 在 $[l, r]$ 中的出现次数。 小B请你帮助他回答询问。

输入格式

第一行三个整数 n, m, k 。

第二行 n 个整数, 表示 小B 的序列。

接下来的 m 行, 每行两个整数 l, r 。

输出格式

输出 m 行, 每行一个整数, 对应一个询问的答案。

输入 #1

```
6 4 3
1 3 2 1 1 3
1 4
2 6
3 5
5 6
```

输出 #1

6
9
5
2

对于 100% 的数据, $1 \leq n, m, k \leq 5 \times 10^4$ 。

思路:

核心思想: 离线分块, 把所有查询按双关键字排序。

暴力处理完第一个查询, 然后每次以 $O(1)$ 的时间复杂度拓展, 处理剩余的查询。

例如第一个查询为 $[l, r]$, 第二个查询为 $[l - 3, r + 4]$, 我们就一步一步地把 l 指针向左移动, 每次移动把拓展的数字加入 `cnt` 数组, 更新答案, 同理操作 r 指针。

怎样才能控制两个指针, 不让它们乱蹦跶呢?

把数列分成 $\sqrt{n}$ 块, 每块的大小 $B = \sqrt{n}$ 。排序时, 第一关键字为左端点 l 所在块编号 l/B , 第二关键字是右端点 r 。这种排序方式会把算法优化到 $O(n\sqrt{n})$ 。后面有证明。

不证明了, 无所谓。

D 图论

D2 狄克斯特拉算法

题目描述

如题, 给出一个有向图, 请输出从某一点出发到所有点的最短路径长度。

输入格式

第一行包含三个整数 n, m, s , 分别表示点的个数、有向边的个数、出发点的编号。

接下来 m 行每行包含三个整数 u, v, w , 表示一条 $u \rightarrow v$ 的, 长度为 w 的边。

输出格式

输出一行 n 个整数, 第 i 个表示 s 到第 i 个点的最短路径, 若不能到达则输出 $2^{31} - 1$ 。

输入 #1

```
4 6 1
1 2 2
2 3 2
2 4 1
1 3 5
3 4 3
1 4 4
```

输出 #1

0 2 4 3

对于 100% 的数据: $1 \leq n \leq 10^4$, $1 \leq m \leq 5 \times 10^5$, $1 \leq u, v \leq n$, $w \geq 0$, $\sum w < 2^{31}$, 保证数据随机。

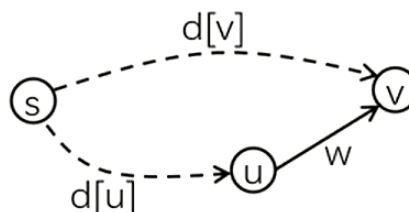
解析: 时间复杂度: $O(n^2)$

Dijkstra (迪杰斯特拉) 算法是基于贪心思想的单源最短路算法。

$e[u]$ 存节点 u 的所有出边的终点和边权。

$d[u]$ 存 u 到源点 s 的最小距离, $vis[u]$ 标记 u 是否出圈。

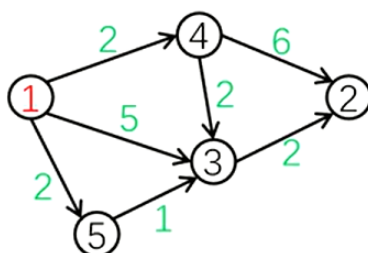
1. 初始时, 所有点都在圈 (集合) 内 $vis=0$, $d[s]=0$, $d[\text{其它点}]=+\infty$;
2. 从圈内选一个距离最小的点 u , 打标记移出圈;
3. 对 u 的所有出边执行松弛操作 (即尝试更新邻点 v 的最小距离);
4. 重复 2, 3 步操作, 直到圈内为空。



5 7 1
1 3 5
1 4 2
1 5 2
4 2 6
4 3 2
5 3 1
3 2 2

建图

1→3, 4, 5
3→2
4→2, 3
5→3



堆优化版:

题目描述

给定一个 n 个点, m 条有向边的带非负权图, 请你计算从 s 出发, 到每个点的距离。

数据保证你能从 s 出发到任意点。

输入格式

第一行为三个正整数 n, m, s 。

第二行起 m 行, 每行三个非负整数 u_i, v_i, w_i , 表示从 u_i 到 v_i 有一条权值为 w_i 的有向边。

输出格式

输出一行 n 个空格分隔的非负整数, 表示 s 到每个点的距离。

输入 #1


```

4 6 1
1 2 2
2 3 2
2 4 1
1 3 5
3 4 3
1 4 4

```

输出 #1

```
0 2 4 3
```

$1 \leq n \leq 10^5;$

$1 \leq m \leq 2 \times 10^5;$

$s = 1;$

$1 \leq u_i, v_i \leq n;$

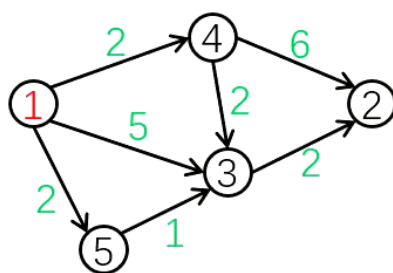
$0 \leq w_i \leq 10^9,$

$0 \leq \sum w_i \leq 10^9.$

Dijkstra 算法的堆优化——用优先队列维护被更新的点的集合。

创建一个pair类型的大根堆 q {-距离, 点}, 把距离取负值, 距离最小的元素最大, 一定在堆顶。

1. 初始化, $\{0, s\}$ 入队, $d[s]=0$, $d[\text{其它点}]=+\infty$;
2. 从队头弹出**距离最小**的点 u , 若 u 扩展过则跳过, 否则打标记;
3. 对 u 的所有出边执行**松弛操作**, 把 $\{-d[v], v\}$ 压入队尾;
4. 重复2, 3步操作, 直到队列为空。



```

{0,1}出队
    {2,4}入队  {2,4}
    {5,3}入队  {2,4} {5,3}
    {2,5}入队  {2,4} {2,5} {5,3}
{2,4}出队
    {8,2}入队  {2,5} {5,3} {8,2}
    {4,3}入队  {2,5} {4,3} {5,3} {8,2}
{2,5}出队
    {3,3}入队  {3,3} {4,3} {5,3} {8,2}
{3,3}出队
    {5,2}入队  {4,3} {5,3} {5,2} {8,2}
{4,3} 出队
{5,3} 出队
{5,2} 出队
{8,2} 出队

```

D4 最短路 Floyd 算法

Floyd(弗洛伊德)算法是动态规划算法，也称插点法。是全源最短路算法。

状态表示

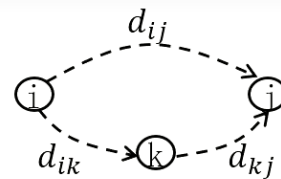
$d[k,i,j]$ 表示从 i 到 j ，且中间只经过节点编号 $1\sim k$ 的最短路径的长度。

状态计算

路径的选择分两类：

(1) 路径不经过 k 点，继承原值： $d[k,i,j]=d[k-1,i,j]$

(2) 路径经过 k 点，松弛操作： $d[k,i,j]=d[k-1,i,k]+d[k-1,k,j]$



```
void floyd(){
    for(int k=1; k<=n; k++)
        for(int i=1; i<=n; i++)
            for(int j=1; j<=n; j++)
                d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
}
```

D5 全源 Johnson 算法

题目描述

给定一个包含 n 个结点和 m 条带权边的有向图，求所有点对间的最短路径长度，一条路径的长度定义为这条路径上所有边的权值和。

注意：边权可能为负，且图中可能存在重边和自环；

输入格式

第 1 行：2 个整数 n, m ，表示给定有向图的结点数量和有向边数量。

接下来 m 行：每行 3 个整数 u, v, w ，表示有一条权值为 w 的有向边从编号为 u 的结点连向编号为 v 的结点。

输出格式

若图中存在负环，输出仅一行 -1 。

若图中不存在负环：

输出 n 行：令 $dis_{i,j}$ 为从 i 到 j 的最短路，在第 i 行输出 $\sum_{j=1}^n j \times dis_{i,j}$ ，注意这个结果可能超过 int 存储范围。

如果不存在从 i 到 j 的路径，则 $dis_{i,j} = 10^9$ ；如果 $i = j$ ，则 $dis_{i,j} = 0$ 。

输入输出样例 #1

输入 #1

```
5 7
1 2 4
1 4 10
2 3 7
4 5 3
4 2 -2
3 4 -3
5 3 4
```

输出 #1

```
128
1000000072
999999978
1000000026
1000000014
```

输入输出样例 #2

输入 #2

```
5 5
1 2 4
3 4 9
3 4 -3
4 5 3
5 3 -2
```

输出 #2

```
-1
```

对于 100% 的数据,

$1 \leq n \leq 3 \times 10^3$, $1 \leq m \leq 6 \times 10^3$, $1 \leq u, v \leq n$, $-3 \times 10^5 \leq w \leq 3 \times 10^5$ 。

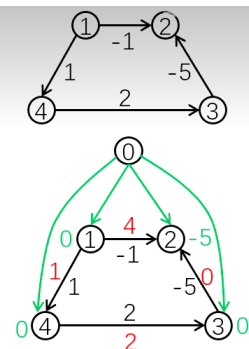
解析: EH不会, 只是默默地放图片说明一切。

➤ 全源最短路算法

1. Floyd 算法, 时间复杂度为 $O(n^3)$
 2. 跑 n 次 Bellman-Ford 算法, 时间复杂度是 $O(n^2m)$
 3. 跑 n 次 Heap-Dijkstra 算法, 时间复杂度是 $O(nm \log m)$
- 第 3 种算法被 Johnson 做了改造, 可以求解带负权边的全源最短路。

➤ Johnson (约翰逊) 算法

1. 新建一个**虚拟源点** 0, 从该点向其他所有点连一条边权为 0 的边, 再用 **spfa 算法** 求出从 0 号点到其他所有点的最短路 $h(i)$ 。
2. 将新图的**边权改造**为 $w(u, v) + h(u) - h(v)$, 这样能确保边权非负。
3. 以每个点为起点, **跑 n 轮 Heap-Dijkstra 算法**, 求出任意两点间最短路。



D7、D8 Prim、Kruscal算法-最小生成树

题目描述

如题, 给出一个无向图, 求出最小生成树, 如果该图不连通, 则输出 **orz**。

输入格式

第一行包含两个整数 N, M , 表示该图共有 N 个结点和 M 条无向边。

接下来 M 行每行包含三个整数 X_i, Y_i, Z_i , 表示有一条长度为 Z_i 的无向边连接结点 X_i, Y_i 。

输出格式

如果该图连通, 则输出一个整数表示最小生成树的各边的长度之和。如果该图不连通则输出 orz。

输入输出样例 #1

输入 #1

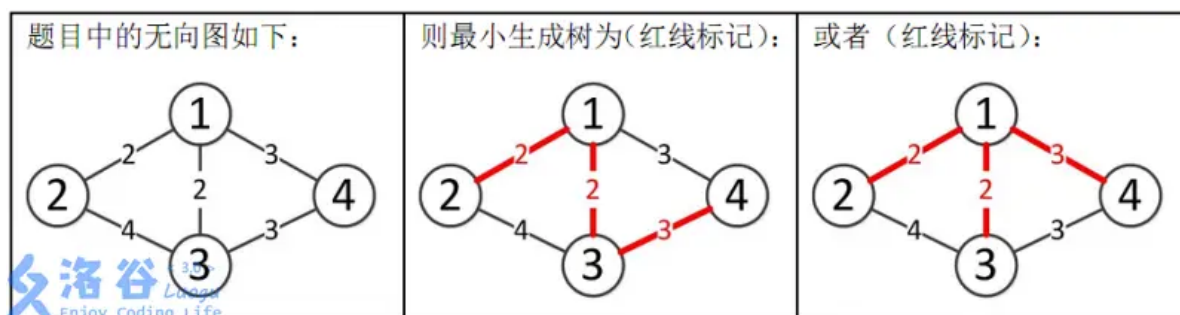
```
4 5
1 2 2
1 3 2
1 4 3
2 3 4
3 4 3
```

输出 #1

```
7
```

对于 100% 的数据: $1 \leq N \leq 5000$, $1 \leq M \leq 2 \times 10^5$, $1 \leq Z_i \leq 10^4$ 。

样例解释:



所以最小生成树的总边权为 $2 + 2 + 3 = 7$ 。

解析:

Prim算法 ($O(n^2)$):

Prim (普里姆) 算法基于贪心思想的最小生成树 (MST) 算法。

$e[u]$ 存 u 点的所有邻边的终点和边权。

$d[u]$ 存 u 点与圈外邻点的最小距离, $vis[u]$ 标记 u 点是否出圈。

算法流程类似 Dijkstra 算法, 不断选距离最小的点出圈, 直到圈内为空。

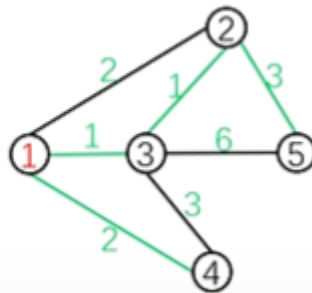
1. 初始化, 所有点都在圈 (集合) 内 $vis=0$, $d[s]=0$, $d[\text{其它点}]=+\infty$ 。
2. 每次从圈内选取一个距离最小的点 u , 打标记移出圈。
3. 对 u 的所有邻点的距离执行更新操作。
4. 重复2, 3步操作, 直到圈内为空。

输入

5 7
1 2 2
1 3 1
1 4 2
2 3 1
2 5 3
3 4 3
3 5 6

建图

1→2,3,4
2→1,3,5
3→1,2,4,5
4→1,3
5→2,3



Kruscal算法 ($O(m\log m)$) :

Kruskal (克鲁斯卡尔) 算法 利用并查集求最小生成树 (MST) 。

$e[i]$ 存第 i 条边的起点、终点与边权。

$fa[x]$ 存 x 点的父节点。

1. 初始化并查集, 把 n 个点放在 n 个独立的集合。
2. 将所有的边按边权从小到大排序 (贪心思想)。
3. 按顺序枚举每一条边, 如果这条边连接的两个点不在同一集合, 就把这条边加入最小生成树, 并且合并这两个集合; 如果这条边连接的两个点在同一集合, 就跳过。
4. 重复执行3, 直到选取了 $n-1$ 条边为止。

D9 倍增算法 (LCA)

题面同D11

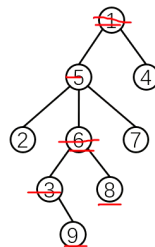
两个节点的**最近公共祖先** (Lowest Common Ancestor, LCA) 就是这两个点的公共祖先里面，离他们最近的那个。

倍增算法是最经典的求 LCA 算法

$dep[u]$ 存 u 点的深度。

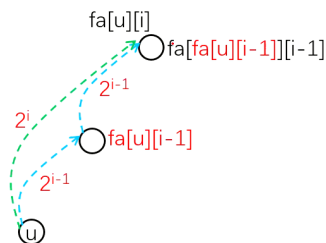
$fa[u][i]$ 存从 u 点向上跳 2^i 层的祖先结点。 $i=0,1,2,3,\dots$

例如，从节点 9 向上跳 2^0 层的祖先是 3，跳 2^1 层的祖先是 6，跳 2^2 层的祖先是 1，跳 2^3 层的祖先是 0(哨兵)。



1. dfs 一遍，创建 ST 表

倍增递推， $fa[u][i]=fa[fa[u][i-1]][i-1]$



	0	1	2	3	...	19
1	0	0	0	0	...	0
5	1	0	0	0	...	0
2	5	1	0	0	...	0
6	5	1	0	0	...	0
3	6	5	0	0	...	0
9	3	6	1	0	...	0

```
const int N=5e5+10;
int n,m,s,a,b;
vector<int> e[N];
int dep[N],fa[N][20];

void dfs(int u, int father){
    dep[u]=dep[father]+1;
    // 向上跳1,2,4...步的祖先节点
    fa[u][0]=father;
    for(int i=1; i<=19; i++){
        fa[u][i]=fa[fa[u][i-1]][i-1];
    }
    for(int v : e[u])
        if(v!=father) dfs(v, u);
}

int lca(int u, int v){
    if(dep[u]<dep[v])swap(u, v);
    // 先跳到同一层
    for(int i=19; i>=0; i--){
        if(dep[fa[u][i]]>=dep[v])
            u=fa[u][i];
    }
    if(u==v) return v;
    // 再跳到LCA的下一层
    for(int i=19; i>=0; i--){
        if(fa[u][i]!=fa[v][i])
            u=fa[u][i], v=fa[v][i];
    }
    return fa[u][0];
}
```

2. 利用 ST 表求 LCA

(1) 第一阶段，将 u, v 跳到同一层

设 u, v 两点的深度之差为 y ，将 y 进行**二进制拆分**，可以将 y 次游标跳跃优化为“ y 的二进制表示所含 1 的个数”次游标跳跃，一定能跳到同一层。

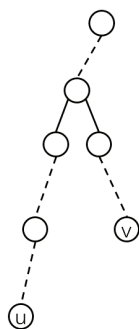
例如， $y=1019$ (0.01111111011)= $512+256+\dots+8+2+1$ ，不越界则跳，共跳 9 次到达。

(2) 第二阶段，将 u, v 一起跳到 LCA 的下一层

从最大的 i 开始循环尝试，一直尝试到 0，最后游标 u, v 一定能停在 LCA 的下一层。

例如， $y=1019$ (0.01111111011)

两游标会跳 $512+256+\dots+8+2=1018$ 层，共跳 8 次到达 LCA 的下一层。



总时间复杂度：
 $O((n+m)\log n)$

```
const int N=5e5+10;
int n,m,s,a,b;
vector<int> e[N];
int dep[N],fa[N][20];

void dfs(int u, int father){
    dep[u]=dep[father]+1;
    // 向上跳1,2,4...步的祖先节点
    fa[u][0]=father;
    for(int i=1; i<=19; i++){
        fa[u][i]=fa[fa[u][i-1]][i-1];
    }
    for(int v : e[u])
        if(v!=father) dfs(v, u);
}

int lca(int u, int v){
    if(dep[u]<dep[v])swap(u, v);
    // 先跳到同一层
    for(int i=19; i>=0; i--){
        if(dep[fa[u][i]]>=dep[v])
            u=fa[u][i];
    }
    if(u==v) return v;
    // 再跳到LCA的下一层
    for(int i=19; i>=0; i--){
        if(fa[u][i]!=fa[v][i])
            u=fa[u][i], v=fa[v][i];
    }
    return fa[u][0];
}
```

D10 Tarjan算法 (离线LCA)

题面同D11

Tarjan(塔扬)算法是一种**离线**算法，巧妙利用**并查集**维护祖先结点。

$e[u]$ 存树边， $e[1]=5$ ， $e[5]=1$ 。
 $query[u]$ 存查询， $query[3]=[4,1]$ ， $query[4]=[3,1]$ 。
 $fa[u]$ 存父节点， $fa[5]=1$ ， $fa[2]=5$ 。
 $vis[u]$ 打标记， $vis[5]=true$ 。
 $ans[i]$ 存查询结果， $ans[1]=1$ ， $ans[2]=5$ 。

1. 从根开始深搜遍历，**入u**时打标记。
2. 枚举u的儿子v，遍历完v的子树，**回u**时把v指向u。
3. 遍历完u的儿子们，**离u**时枚举以u为起点的查询，若终点v被搜过，则查找v的根，即u,v的LCA，答案记入ans[]。
4. 递归遍历完整颗树，得到全部查询答案。

9 2 1

1 5

1 4

4 9

5 2

5 6

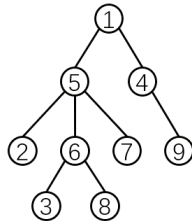
5 7

6 3

6 8

3 4

7 8



```
for(int i=1; i<n; i++){
    scanf("%d%d",&a,&b);
    e[a].push_back(b);
    e[b].push_back(a);
}
for(int i=1; i<=m; i++){
    scanf("%d%d",&a,&b);
    query[a].push_back({b,i});
    query[b].push_back({a,i});
}
for(int i=1; i<=N; i++)fa[i]=i;
```

```
vector<int> e[N];
vector<pair<int,int>> query[N];
int fa[N],vis[N],ans[M];

int find(int u){
    if(u==fa[u]) return u;
    return fa[u]=find(fa[u]);
}

void tarjan(int u){
    vis[u]=true;//入u时 标记u
    for(auto v : e[u]){
        if(!vis[v]){
            tarjan(v);
            fa[v]=u;//回u时 v指向u
        }
    }
    //离u时 枚举LCA
    for(auto q : query[u]){
        int v=q.first,i=q.second;
        if(vis[v])ans[i]=find(v);
    }
}
```

D11 树链剖分 (最近公共祖先 LCA)

题目描述

如题，给定一棵有根多叉树，请求出指定两个点直接最近的公共祖先。

输入格式

第一行包含三个正整数 N, M, S ，分别表示树的结点个数、询问的个数和树根结点的序号。

接下来 $N - 1$ 行每行包含两个正整数 x, y ，表示 x 结点和 y 结点之间有一条直接连接的边（数据保证可以构成树）。

接下来 M 行每行包含两个正整数 a, b ，表示询问 a 结点和 b 结点的最近公共祖先。

输出格式

输出包含 M 行，每行包含一个正整数，依次为每一个询问的结果。

输入输出样例 #1

输入 #1

```
5 5 4
3 1
2 4
5 1
1 4
2 4
3 2
3 5
1 2
4 5
```

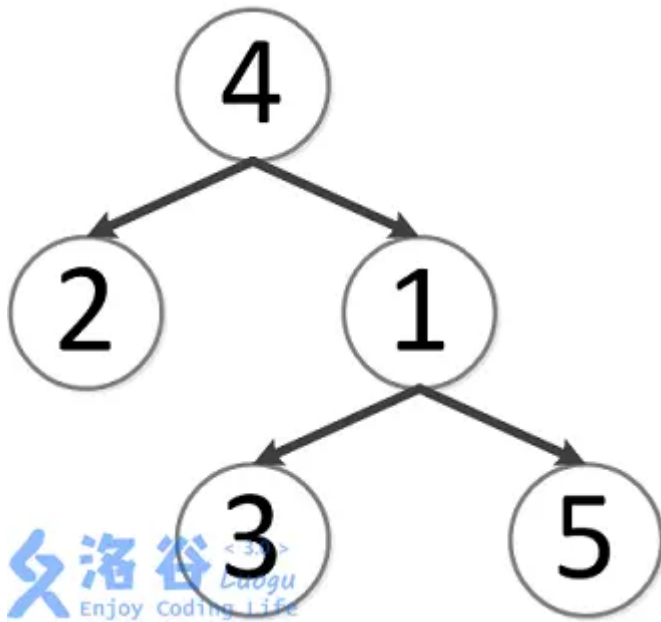
输出 #1

4
4
1
4
4

对于 100% 的数据, $1 \leq N, M \leq 500000$, $1 \leq x, y, a, b \leq N$, 不保证 $a \neq b$ 。

样例说明:

该树结构如下:



第一次询问: 2, 4 的最近公共祖先, 故为 4。

第二次询问: 3, 2 的最近公共祖先, 故为 4。

第三次询问: 3, 5 的最近公共祖先, 故为 1。

第四次询问: 1, 2 的最近公共祖先, 故为 4。

第五次询问: 4, 5 的最近公共祖先, 故为 4。

故输出依次为 4, 4, 1, 4, 4。

解析:

树链剖分

重儿子：父结点的所有儿子中子树结点数目最多的结点

轻儿子：父结点中除重儿子以外的儿子

重边：父结点和重儿子连成的边

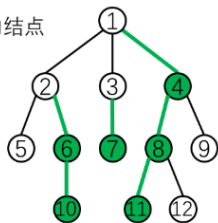
轻边：父结点和轻儿子连成的边

重链：由多条重边连接而成的路径

1. 整棵树会被剖分成若干条重链。

2. 轻儿子一定是每条重链的顶点。

3. 任意一条路径被切分成不超过 $\log n$ 条链。



数组

fa[u]: 存 u 的父结点

dep[u]: 存 u 的深度

son[u]: 存 u 的重儿子

sz[u]: 存以 u 为根的子树的结点数

top[u]: 存 u 所在重链的顶点

流程

1. 第一遍 dfs, 搞出 fa, dep, son 数组
2. 第二遍 dfs, 搞出 top 数组
3. 让两个游标沿着各自的重链向上跳, 跳到同一条重链上时, 深度较小的那个游标所指向的点, 就是 LCA

dfs1()

```
son[2]=5  
son[6]=10  
son[2]=6  
son[1]=2  
son[3]=7  
son[8]=11  
son[4]=8  
son[1]=4
```

```
lca(10, 12)  
u=12, v=10  
u=8  
u=10, v=8  
u=1 re 1
```

dfs2()

```
1. top[1]=1  
top[4]=1  
top[8]=1  
top[11]=1  
2. top[12]=12  
3. top[9]=9  
4. top[2]=2  
top[6]=2  
top[10]=2  
5. top[5]=5  
6. top[3]=3  
top[7]=3
```

```
vector<int> e[N];  
int fa[N], dep[N], son[N], sz[N];  
int top[N];  
void dfs1(int u, int father){ //搞fa, dep, son  
    fa[u]=father, dep[u]=dep[father]+1, sz[u]=1;  
    for(int v:e[u]){  
        if(v==father) continue;  
        dfs1(v, u);  
        sz[u]+=sz[v];  
        if(sz[son[u]]<sz[v]) son[u]=v;  
    }  
}  
void dfs2(int u, int t){ //搞top  
    top[u]=t; //记录链头  
    if(!son[u]) return; //无重儿子返回  
    dfs2(son[u], t); //搜重儿子  
    for(int v:e[u]){  
        if(v==fa[u] || v==son[u]) continue;  
        dfs2(v, v); //搜轻儿子  
    }  
}  
int lca(int u, int v){  
    while(top[u]!=top[v]){  
        if(dep[top[u]]<dep[top[v]]) swap(u, v);  
        u=fa[top[u]];  
    }  
    return dep[u]<dep[v]?u:v; }  
//总 O(n + mlogn)
```

D13 树上距离

题目描述

给定一棵 n 个点的树, Q 个询问, 每次询问点 x 到点 y 两点之间的距离。

输入格式

第一行一个正整数 n , 表示这棵树有 n 个节点;

接下来 $n-1$ 行, 每行两个整数 x, y 表示 x, y 之间有一条连边;

然后一个整数 Q , 表示有 Q 个询问;

接下来 Q 行每行两个整数 x, y 表示询问 x 到 y 的距离。

输出格式

输出 Q 行, 每行表示每个询问的答案。

样例

输入

```
6  
1 2  
1 3  
2 4  
2 5  
3 6  
2  
2 6  
5 6
```

复制

输出

```
3  
4
```

复制

数据范围与提示

对于全部数据, $1 \leq n, Q \leq 10^5, 1 \leq x, y \leq n$ 。

解析: 无, LCA 板子题举例而已。

E-动态规划

E4 最长上升子序列（二分优化）

题目描述

给出一个由 n ($n \leq 5000$) 个不超过 10^6 的正整数组成的序列。请输出这个序列的**最长上升子序列**的长度。

最长上升子序列是指，从原序列中**按顺序**取出一些数字排在一起，这些数字是**逐渐增大**的。

输入格式

第一行，一个整数 n ，表示序列长度。

第二行有 n 个整数，表示这个序列。

输出格式

一个整数表示答案。

输入输出样例 #1

输入 #1

```
6
1 2 4 1 3 4
```

输出 #1

```
4
```

解析：时间复杂度： $O(n \log n)$

考虑新进来一个元素 $a[i]$ ：

（1）**大则添加**：如果 $a[i]$ 大于 $b[len]$ ，直接让 $b[++len] = a[i]$ 。即 b 数组的长度增加1，而且添加了一个元素。

（2）**小则替换**：如果 $a[i]$ 小于或等于 $b[len]$ ，就用 $a[i]$ 替换掉 b 数组中**第一个大于或等于 $a[i]$ 的元素**。

假设第一个大于 $a[i]$ 的元素是 $b[j]$ ，那么用 $a[i]$ 换掉 $b[j]$ 后，会使得 $b[1 \dots j]$ 这个上升子序列的结尾元素更小。对于一个上升子序列，其**结尾元素越小，越有利于续接其它元素**，也就越可能变得更长。

E5 最长公共子序列

题目描述

给出一个长度为 n ($n \leq 5000$) 的字符串序列。请输出这个序列的**最长公共子序列**及其长度。

输入格式

输入两个字符串（代码里忽略了这个）。

输出格式

输出分为两行，

第一行输出一个整数表示最长公共子序列的长度，

第二行输出这个最长公共子序列。

解析：时间复杂度： $O(n^2)$

设 $f[i][j]$ 表示序列 $a[1 \dots i]$ 和 $b[1 \dots j]$ 的最长公共子序列长度。
现在考察**末尾元素** $a[i]$ 与 $b[j]$ 是否在公共子序列中：

(1) 若 $a[i] = b[j]$ ，则 $a[i]$ 与 $b[j]$ 在公共子序列中，
 $a[1 \dots i-1, i] \quad a[1 \dots i-1] + a[i]$
 $b[1 \dots j-1, j] \quad b[1 \dots j-1] + b[j]$
所以， $f[i][j] = f[i-1][j-1] + 1$

A D A B E C
 ↑
D B D C A
 ↑
 j

(2) 若 $a[i] \neq b[j]$ ，且 $a[i]$ 不在公共子序列中，则可去掉 $a[i]$ ，
 $a[1 \dots i-1, i] \quad a[1 \dots i-1]$
 $b[1 \dots j-1, j] \quad b[1 \dots j-1, j]$
所以， $f[i][j] = f[i-1][j]$

A D A B E C
 ↑
D B D C A
 ↑
 j

(3) 若 $a[i] \neq b[j]$ ，且 $b[j]$ 不在公共子序列中，则可去掉 $b[j]$ ，
 $a[1 \dots i-1, i] \quad a[1 \dots i-1, i]$
 $b[1 \dots j-1, j] \quad b[1 \dots j-1]$
所以， $f[i][j] = f[i][j-1]$

A D A B E C
 ↑
D B D C A
 ↑
 j

递推式（状态转移方程）：

$$f[i][j] = \begin{cases} f[i-1][j-1] + 1, & a[i] = b[j] \\ \max(f[i-1][j], f[i][j-1]), & a[i] \neq b[j] \end{cases}$$

p数组记录转移方向，用来还原子序列。

		j	0	1	2	3	4	5
		i		D	B	D	C	A
0			0	0	0	0	0	0
1	A		0	↑ 0	↑ 0	↑ 0	↑ 0	↖ 1
2	D		0	↖ 1	← 1	↖ 1	← 1	↑ 1
3	A		0	↑ 1	↑ 1	↑ 1	↑ 1	↖ 2
4	B		0	↑ 1	↖ 2	← 2	← 2	↑ 2
5	E		0	↑ 1	↑ 2	↑ 2	↑ 2	↑ 2
6	C		0	↑ 1	↑ 2	↑ 2	↖ 3	← 3

E6 最长公共子串

题目描述

给出一个长度为 n ($n \leq 5000$) 的字符串序列。请输出这个序列的**最长公共子串**及其长度。

输入格式

输入两个字符串（代码里忽略了这个）。

输出格式

输出一个整数表示最长公共子串的长度，

解析：时间复杂度： $O(n^2)$

A B A C C B

A A C C A B

↑

i

↑

j

只有当两字符串中的字符连续相等时，公共子串的长度才不断增加，否则清零。

状态变量：
用 $f[i][j]$ 表示以 $a[i]$ 和 $b[j]$ 为结尾的公共子串的长度。

递推式：

- (1) 若 $a[i] == b[j]$ ，则可以构成公共子串，
 $f[i][j] = f[i-1][j-1] + 1$;
- (2) 若 $a[i] != b[j]$ ，则不能构成公共子串，
 $f[i][j] = 0$

E8 01背包

题目描述

有 N 件物品和一个容量为 M 的背包。第 i 件物品的重量是 W_i ，价值是 D_i 。求解将哪些物品装入背包可使这些物品的重量总和不超过背包容量，且价值总和最大。

输入格式

第一行：物品个数 N 和背包大小 M 。

第二行至第 $N + 1$ 行：第 i 个物品的重量 W_i 和价值 D_i 。

输出格式

输出一行最大价值。

输入输出样例 #1

输入 #1

```
4 6
1 4
2 6
3 12
2 7
```

输出 #1

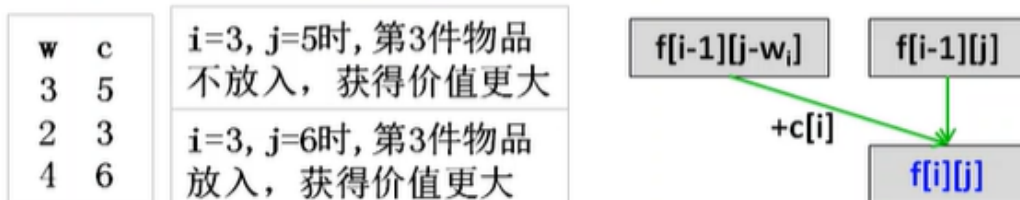
```
23
```

解析：时间复杂度： $O(mn)$ ，空间复杂度方面，可以使用一维数组+逆向查找优化。

状态变量: $f[i][j]$ 表示前 i 件物品放入容量为 j 的背包的最大价值。

当前背包容量为 j , 我们要考虑第 i 件物品能否放入? 是否放入?

1. 如果当前背包容量 $j < w[i]$, 不能放入, 则 $f[i][j] = f[i-1][j]$
2. 如果当前背包容量 $j \geq w[i]$, 能放入, 但比较代价
 - (1) 如果第 i 件物品不放入背包, 则 $f[i][j] = f[i-1][j]$
 - (2) 如果第 i 件物品放入背包, 则 $f[i][j] = f[i-1][j-w[i]] + c[i]$如果第 i 件物品放入背包, 则背包容量还剩 $j-w[i]$, 所以要取前 $i-1$ 件物品放入背包剩余容量 $j-w[i]$ 所获得的最大价值 $f[i-1][j-w[i]]$ 。



状态转移方程:

$f[i][j] = f[i-1][j], (j < w[i])$

$f[i][j] = \max(f[i-1][j], f[i-1][j-w[i]] + c[i]), (j \geq w[i])$

E9 完全背包

题目不码了, 跟上题一样, 只不过是把只能拿一件物品改为可以拿任意件。

解析: 时间复杂度: $O(mn)$, 空间复杂度方面, 可以使用一维数组优化。

状态变量: $f[i][j]$ 表示前 i 件物品放入容量为 j 的背包的最大价值。

当前背包容量为 j , 我们要考虑第 i 件物品能否放入? 是否放入?

1. 当前背包容量 $j < w[i]$, 不能放入, 则 $f[i][j] = f[i-1][j]$
2. 当前背包容量 $j \geq w[i]$, 能放入, 但比较代价
 - (1) 若第 i 件物品不放入背包, 则 $f[i][j] = f[i-1][j]$
 - (2) 若第 i 件物品放入背包, 则 $f[i][j] = f[i][j-w[i]] + c[i]$对于前 i 件物品, 背包容量为 $j-w[i]$ 时可能已经放入了第 i 件物品, 容量为 j 时还可以再放入第 i 件物品, 所以用 $f[i][j-w[i]]$ 更新 $f[i][j]$

E10 多重背包

题目描述

小 FF 对洞穴里的宝物进行了整理, 他发现每样宝物都有一件或者多件。他粗略估算了下每样宝物的价值, 之后开始了宝物筛选工作: 小 FF 有一个最大载重为 W 的采集车, 洞穴里总共有 n 种宝物, 每种宝物的价值为 v_i , 重量为 w_i , 每种宝物有 m_i 件。小 FF 希望在采集车不超载的前提下, 选择一些宝物装进采集车, 使得它们的价值和最大。

输入格式

第一行为两个整数 n 和 W ，分别表示宝物种数和采集车的最大载重。

接下来 n 行每行三个整数 v_i, w_i, m_i 。

输出格式

输出仅一个整数，表示在采集车不超载的情况下收集的宝物的最大价值。

输入输出样例 #1

输入 #1

```
4 20
3 9 3
5 9 1
9 4 2
8 1 3
```

输出 #1

```
47
```

对于 100% 的数据, $n \leq \sum m_i \leq 10^5$, $0 \leq W \leq 4 \times 10^4$, $1 \leq n \leq 100$ 。

二进制拆分思想：

将第 i 种物品拆分成若干件物品，每件物品的体积和价值乘以一个拆分系数 $(1, 2^1, 2^2 \dots 2^{k-1}, s_i - 2^{k-1})$ ，就可以转化成 01 背包的物品求解。

例如， $s_i=12$ ，拆分系数为 1, 2, 4, 5，转化成 4 件 01 背包的物品：
 $(v_i, w_i), (2v_i, 2w_i), (4v_i, 4w_i), (5v_i, 5w_i)$

E11 滑动窗口

题目描述

有一个长为 n 的序列 a ，以及一个大小为 k 的窗口。现在这个从左边开始向右滑动，每次滑动一个单位，求出每次滑动后窗口中的最大值和最小值。

例如，对于序列 $[1, 3, -1, -3, 5, 3, 6, 7]$ 以及 $k = 3$ ，有如下过程：

窗口位置								最小值	最大值
[1	3	-1]	-3	5	3	6	7	-1	3
1	[3	-1	-3]	5	3	6	7	-3	3
1	3	[-1	-3	5]	3	6	7	-3	5
1	3	-1	[-3	5	3]	6	7	-3	5
1	3	-1	-3	[5	3	6]	7	3	6
1	3	-1	-3	5	[3	6	7]	3	7

输入格式

输入一共有两行，第一行有两个正整数 n, k 。

第二行 n 个整数，表示序列 a

输出格式

输出共两行，第一行为每次窗口滑动的最小值

第二行为每次窗口滑动的最大值

输入输出样例 #1

输入 #1

```
8 3
1 3 -1 -3 5 3 6 7
```

输出 #1

```
-1 -3 -3 -3 3 3
3 3 5 5 6 7
```

对于 100% 的数据, $1 \leq k \leq n \leq 10^6$, $a_i \in [-2^{31}, 2^{31})$ 。

时间复杂度: $O(n)$

```
// 维护窗口最小值
int h=1, t=0;
for(int i=1; i<=n; i++){
    while(h<=t && a[q[t]]>=a[i]) t--;
    q[++t]=i;
    if(q[h]<i-k+1) h++;
    if(i>=k) printf("%d ", a[q[h]]);
}
```

每个元素最多进队和出队各一次, 时间复杂度为 $O(n)$

E17 树形DP

EH声明: 代码使用了链表储存结构, 感觉不是很符合个人习惯, 可以酌情更换方法。

题目描述

某大学有 n 个职员, 编号为 $1 \dots n$ 。

他们之间有从属关系, 也就是说他们的关系就像一棵以校长为根的树, 父结点就是子结点的直接上司。

现在有个周年庆宴会, 宴会每邀请来一个职员都会增加一定的快乐指数 r_i , 但是呢, 如果某个职员的直接上司来参加舞会了, 那么这个职员就无论如何也不肯来参加舞会了。

所以, 请你编程计算, 邀请哪些职员可以使快乐指数最大, 求最大的快乐指数。

输入格式

输入的第一行是一个整数 n 。

第 2 到第 $(n + 1)$ 行, 每行一个整数, 第 $(i + 1)$ 行的整数表示 i 号职员的快乐指数 r_i 。

第 $(n + 2)$ 到第 $2n$ 行, 每行输入一对整数 l, k , 代表 k 是 l 的直接上司。

输出格式

输出一行一个整数代表最大的快乐指数。

输入输出样例 #1

输入 #1

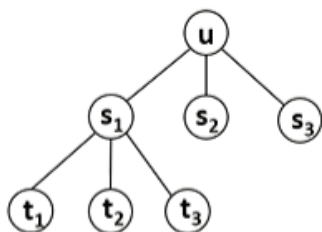
```
7
1
1
1
1
1
1
1
1
1 3
2 3
6 4
7 4
4 5
3 5
```

输出 #1

```
5
```

对于 100% 的数据，保证 $1 \leq n \leq 6 \times 10^3$ ， $-128 \leq r_i \leq 127$ ， $1 \leq l, k \leq n$ ，且给出的关系一定是一棵树。

解析：



考虑一颗以u为根节点的子树，这颗子树的快乐指数应该是u的函数，并且分两种情况：选u和不选u。

状态表示：

$f[u][1]$ 表示以u为根节点的子树并且包括u的总快乐指数，
 $f[u][0]$ 表示以u为根节点的子树并且不包括u的总快乐指数。

状态计算：

记点u的子节点是s，

1.选u， $f[u][1] += \sum f[s][0]$

2.不选u， $f[u][0] += \sum \max(f[s][1], f[s][0])$

从根节点u开始dfs一遍即可，
从根到叶深搜，从叶到根返回时，
做DP，累加f值。

```
void dfs(int u) // 深搜节点+后序 DP
{
    f[u][1] = w[u]; // 选u的快乐指数
    for(int i=0; i<b[u]; i++){
        int son = a[u][i]; // 取u的子节点
        dfs(son);
        f[u][0] += max(f[son][0], f[son][1]);
        f[u][1] += f[son][0];
    }
}
```

F-字符串

EH过度劳累，此时的san值有点低，精神崩溃中，写水一点吧。

F3 KMP算法

题目描述

给出两个字符串 s_1 和 s_2 , 若 s_1 的区间 $[l, r]$ 子串与 s_2 完全相同, 则称 s_2 在 s_1 中出现了, 其出现位置为 l 。

现在请你求出 s_2 在 s_1 中所有出现的位置。

定义一个字符串 s 的 border 为 s 的一个**非 s 本身**的子串 t , 满足 t 既是 s 的前缀, 又是 s 的后缀。

对于 s_2 , 你还要求出对于其每个前缀 s' 的最长 border t' 的长度。

输入格式

第一行为一个字符串, 即为 s_1 。

第二行为一个字符串, 即为 s_2 。

输出格式

首先输出若干行, 每行一个整数, **按从小到大的顺序**输出 s_2 在 s_1 中出现的位置。

最后一行输出 $|s_2|$ 个整数, 第 i 个整数表示 s_2 的长度为 i 的前缀的最长 border 长度。

输入输出样例 #1

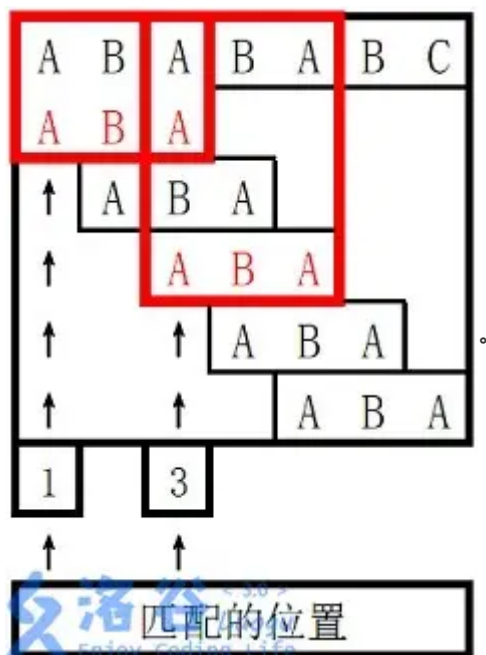
输入 #1

```
ABABABC
ABA
```

输出 #1

```
1
3
0 0 1
```

样例 1 解释



对于 s_2 长度为 3 的前缀 ABA , 字符串 A 既是其后缀也是其前缀, 且是最长的, 因此最长 border 长度为 1。

对于全部的测试点, 保证 $1 \leq |s_1|, |s_2| \leq 10^6$, s_1, s_2 中均只含大写英文字母。

解析:

给定一个模式串P和一个主串S，求模式串P在主串S中出现的位置。（字符串的下标均从1开始）

1. 取**最长**的相等前后缀，可以保证不漏解。
2. 通过模式串前后缀的**自我匹配**的长度，计算 **next 函数**，给 j 指针打一张表，失配时就跳到 next[j] 的位置继续匹配。

next函数

next[i] 表示模式串 P[1,i] 中**相等前后缀**的**最长**长度。

难点主要是next的计算:

```
ne[1] = 0;
for(int i = 2, j = 0; i <= n; i++){
    while(j && P[i] != P[j+1]) j = ne[j];
    if(P[i] == P[j+1]) j++;
    ne[i] = j;
}
```

双指针: i 扫描模式串, j 扫描前缀。

初始化, ne[1]=0, i=2, j=0。

每轮 for 循环, i 向右走一步。

1. 若 $P[i] \neq P[j+1]$, 让 j 回跳到能匹配的位置, 如果找不到能匹配的位置, j 回跳到 0。
2. 若 $P[i] = P[j+1]$, 让 j+1, **指向匹配前缀的末尾**。
3. next[i] 等于 j 的值。

```
i=2 j=1 ne[2]=1
i=3 *j=0
i=3 j=0 ne[3]=0
i=4 j=1 ne[4]=1
i=5 j=2 ne[5]=2
i=6 j=3 ne[6]=3
i=7 j=4 ne[7]=4
i=8 j=5 ne[8]=5
i=9 *j=2
i=9 *j=1
i=9 j=2 ne[9]=2
i=10 *j=1
i=10 j=2 ne[10]=2
```

i=2
a a b a a b a a a a
a a b a a b a a a a
j=0

i=8
a a b a a b a a a a
a a b a a b a a a a
j=5

i=9
a a b a a b a a a a
a a b a a b a a a a
j=5



预判: 进可攻, 退可守。回跳: 退而求其次的过程。

F6 Trie字典树

代码对应的题目:

输入I表示插入, 否则是查询, 查询时输出即可:

Trie 是一种能够快速插入和查询字符串的**多叉树**结构。节点的编号各不相同, 根节点编号为 0, 其他节点用来标识路径, 还可以标记单词插入的次数。边表示字符。

Trie 维护字符串的集合, 支持两种操作:

1. 向集合中**插入**一个字符串, `void insert(char *s)`
2. 在集合中**查询**一个字符串, `int query(char *s)`

建树过程:

建字典树

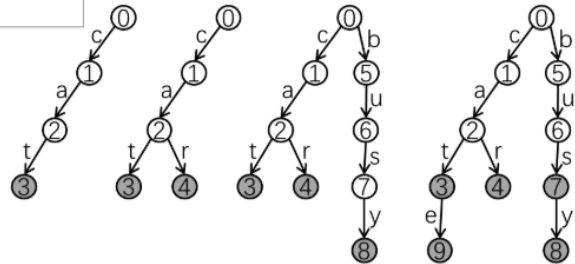
儿子数组 $ch[p][j]$ 存储从节点 p 沿着 j 这条边走到的子节点。
边为26个小写字母(a~z)对应的映射值0~25，
每个节点最多可以有26个分叉。
例如， $ch[0][2]=1$ ， $ch[1][0]=2$ ， $ch[2][19]=3$ 。
计数数组 $cnt[p]$ 存储以节点 p 结尾的单词的插入次数。
节点编号 idx 用来给节点编号。

1. 空 Trie 仅有一个根节点，编号为0。 $ch[][]$ 初始值全部为 0。
2. 从根开始插，枚举字符串的每个字符，
如果有儿子，则 p 指针走到儿子，
如果没儿子，则先创建儿子， p 指针再走到儿子。
3. 在单词结束点记录插入次数。

```
char s[N];
int ch[N][26], cnt[N], idx;

void insert(char *s){
    int p = 0;
    for(int i = 0; s[i]; i++){
        int j = s[i] - 'a'; //字母映射
        if(!ch[p][j]) ch[p][j] = ++idx;
        p = ch[p][j];
    }
    cnt[p]++; //插入次数
}
```

依次插入“cat”“car”“busy”“cate”“bus”“car”



F7 最大异或对

题目：

给定 N 个整数 $a_1, a_2, \dots, a_N$,

任选两个进行异或运算，得到的结果最大是多少？

$1 \leq N \leq 10^5, 0 \leq a_i < 2^{31}$

思路：

异或运算 $\rightarrow$ 二进制位 $\rightarrow N$ 个整数均转化为二进制数表示

$\rightarrow$ 二进制是“01”构成的串 $\rightarrow$ 构造 Tire 树 $\rightarrow$ 在树枝上进行异或运算。

说明：

用 Trie 存单词，由26个小写字母构造的 Trie，是一颗26叉树，深度为最长单词的长度。

用 Trie 存整数，由整数的十进制位构造的 Trie，是一颗10叉树，深度为10层。

用 Trie 存整数，由整数的二进制位构造的 Trie，是一颗二叉树，深度为31层。

EH是懒鬼，这段代码没有测验，直接AI写的，管他呢。。。

G-数学

G1 快速幂

题目描述

给你三个整数 a, b, p ，求 $a^b \bmod p$ 。

输入格式

输入只有一行三个整数，分别代表 a, b, p 。

输出格式

输出一行一个字符串 $a^b \bmod p=s$ ，其中 a, b, p 分别为题目给定的值， s 为运算结果。

输入 #1

2 10 9

输出 #1

$$2^{10} \bmod 9 = 7$$

G2 高精度快速幂

题目描述

形如 $2^P - 1$ 的素数称为麦森数，这时 P 一定也是个素数。但反过来不一定，即如果 P 是个素数， $2^P - 1$ 不一定也是素数。到 1998 年底，人们已找到了 37 个麦森数。最大的一个是 $P = 3021377$ ，它有 909526 位。麦森数有许多重要应用，它与完全数密切相关。

任务：输入 $P(1000 < P < 3100000)$ ，计算 $2^P - 1$ 的位数和最后 500 位数字（用十进制高精度数表示）

输入格式

文件中只包含一个整数 $P(1000 < P < 3100000)$

输出格式

第一行：十进制高精度数 $2^P - 1$ 的位数。

第 2 ~ 11 行: 十进制高精度数 $2^P - 1$ 的最后 500 位数字。(每行输出 50 位, 共输出 10 行, 不足 500 位时高位补 0)

不必验证 $2^P - 1$ 与 P 是否为素数。

输入 #1

1279

输出 #1

[illegible]

备注：本质上就是高精度乘法+快速幂

G5 gcd及lcm问题

题目描述

输入两个正整数 x_0, y_0 , 求出满足下列条件的 P, Q 的个数:

1. P, Q 是正整数。
2. 要求 P, Q 以 x_0 为最大公约数, 以 y_0 为最小公倍数。

试求: 满足条件的所有可能的 P, Q 的个数。

输入格式

一行两个正整数 x_0, y_0 。

输出格式

一行一个数, 表示求出满足条件的 P, Q 的个数。

输入输出样例 #1

输入 #1

```
3 60
```

输出 #1

```
4
```

对于 100% 的数据, $2 \leq x_0, y_0 \leq 10^5$ 。

G3 矩阵快速幂

Luogu P3390 【模板】矩阵快速幂

给定 $n \times n$ 的矩阵 A , 求 A^k 。

输入格式

第一行两个整数 n, k 接下来 n 行, 每行 n 个整数, 第 i 行的第 j 的数表示 $A_{i,j}$ 。

输出格式

输出 A^k

共 n 行, 每行 n 个数, 第 i 行第 j 个数表示 $(A^k)_{i,j}$, 每个元素对 10^9+7 取模。

输入样例

```
2 3
2 3
1 2
2 1
```

输出样例

```
13 14
14 13
```

数据范围

对于 100% 的数据: $1 \leq n \leq 100, 0 \leq k \leq 10^{12}, |A_{i,j}| \leq 1000$

这个题可能会和DP放在一起考~, 原理不解释了, 本质上也是快速幂, 时间复杂度: $O(n^3 \log k)$

G8 线性筛质数

题目描述

如题，给定一个范围 n ，有 q 个询问，每次输出第 k 小的素数。

输入格式

第一行包含两个正整数 n, q ，分别表示查询的范围和查询的个数。

接下来 q 行每行一个正整数 k ，表示查询第 k 小的素数。

输出格式

输出 q 行，每行一个正整数表示答案。

输入输出样例 #1

输入 #1

```
100 5
1
2
3
4
5
```

输出 #1

```
2
3
5
7
11
```

对于 100% 的数据， $n = 10^8$ ， $1 \leq q \leq 10^6$ ，保证查询的素数不大于 n 。

解析：时间复杂度： $O(n)$

线性筛法（又称欧拉筛法）

从小到大枚举每个数

1. 如果当前数没划掉，必定是质数，记录该质数

2. 枚举已记录的质数（如果合数已越界则中断）

(1) 合数未越界，则划掉合数

(2) 条件 $i \% p == 0$ ，保证合数只被最小质因子划掉

若 i 是质数，则最多枚举到自身中断

若 i 是合数，则最多枚举到自身的最小质数中断

$n=30$

```
i = 2: p{2}; v{4}; (2%2) break
i = 3: p{3}; v{6,9}; (3%3) break
i = 4: v{8}; (4%2) break
i = 5: p{5}; v{10,15,25}; (5%5) break
i = 6: v{12}; (6%2) break
i = 7: p{7}; v{14,21}; (35>30)
i = 8: v{16}; (8%2) break
i = 9: v{18,27}; (9%3) break
i=10: v{20}; (10%2) break
i=11: p{11}; v{22}; (33>30)
i=12: v{24}; (12%2) break
.....
i=16: (32>30)
```

```
const int N = 100000010;
int vis[N]; //划掉合数
int prim[N]; //记录质数
int cnt; //质数个数

void get_prim(int n){ //线性筛法
    for(int i=2; i<=n; i++){
        if(!vis[i]) prim[++cnt] = i;
        for(int j=1; 1ll*i*prim[j]<=n; j++){
            vis[i*prim[j]] = 1;
            if(i%prim[j] == 0) break;
        }
    }
}
```

整除中断

越界中断

G9 筛法求欧拉函数

欧拉函数的定义

1~n中与n互质的数的个数称为欧拉函数，记为 $\varphi(n)$

欧拉函数的性质

1. 若p是质数，则 $\varphi(p) = p - 1$
2. 若p是质数，则 $\varphi(p^k) = (p - 1)p^{k-1}$
3. 积性函数：若 $\gcd(m, n) = 1$ ，则 $\varphi(mn) = \varphi(m)\varphi(n)$

欧拉函数的计算公式

由唯一分解定理 $n = \prod_{i=1}^s p_i^{\alpha_i} = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_s^{\alpha_s}$,

得 $\varphi(n) = n \times \prod_{i=1}^s \frac{p_i - 1}{p_i} = n \times \frac{p_1 - 1}{p_1} \times \frac{p_2 - 1}{p_2} \times \cdots \times \frac{p_s - 1}{p_s}$

筛法求欧拉函数

若i是质数， $\text{phi}[i] = i - 1$ 。

在线性筛中，每个合数m都是被最小的质因子筛掉的。

设 p_j 是m的最小质因子，则m通过 $m = p_j \times i$ 筛掉。

(1) 若i能被 p_j 整除，则i包含了m的所有质因子。

$$\varphi(m) = m \times \prod_{k=1}^s \frac{p_k - 1}{p_k} = p_j \times i \times \prod_{k=1}^s \frac{p_k - 1}{p_k} = p_j \times \varphi(i)$$

例， $\varphi(12) = \varphi(2 \times 6) = 2 \times \varphi(6)$

(2) 若i不能被 p_j 整除，则i和 p_j 是互质的。

$$\varphi(m) = \varphi(p_j \times i) = \varphi(p_j) \times \varphi(i) = (p_j - 1) \times \varphi(i)$$

例， $\varphi(75) = \varphi(3 \times 25) = (3 - 1) \times \varphi(25)$

```
const int N = 1000010;
int p[N], vis[N], cnt;
int phi[N];

void get_phi(int n){ //筛法求欧拉函数
    phi[1] = 1;
    for(int i=2; i<=n; i++){
        if(!vis[i]){
            p[cnt++] = i;
            phi[i] = i-1;
        }
        for(int j=0; i*p[j]<=n; j++){
            int m = i*p[j];
            vis[m] = 1;
            if(i%p[j] == 0){
                phi[m] = p[j]*phi[i];
                break;
            }
            else
                phi[m] = (p[j]-1)*phi[i];
        }
    }
}
```

它的本质上是线性筛，但由于欧拉函数后续太好用了，这里还是写一下吧。

G10 筛法求因数个数

问题：

给一个数n ($n \leq 1 \times 10^6$)，输出1~n每个数的约数个数。

约数个数定理

若 $n = \prod_{i=1}^s p_i^{\alpha_i}$ ，则 $d(n) = \prod_{i=1}^s (\alpha_i + 1)$ 。

证明： $p_i^{\alpha_i}$ 的约数有 $p_i^0, p_i^1, \dots, p_i^{\alpha_i}$ 共 $(\alpha_i + 1)$ 个，
根据乘法原理， $d(n) = \prod_{i=1}^s (\alpha_i + 1)$ 。

筛法求约数个数

若i是质数， $a[i] = 1, d[i] = 2$ 。

在线性筛中，每个合数m都是被最小的质因子筛掉的。

设 p_j 是m的最小质因子，则m通过 $m = p_j \times i$ 筛掉。

(1) 若i能被 p_j 整除，则 p_j 一定是i的最小质因子。

$$a[m] = a[i] + 1;$$

$$d[i] = (a[i] + 1) \times \dots, d[m] = (a[m] + 1) \times \dots$$

(2) 若i不能被 p_j 整除，则i不包含质因子 p_j 。

$$a[m] = 1, d[m] = d[i] \times (1 + 1)$$

```
const int N = 1000010;
int p[N], vis[N], cnt;
int a[N]; //a[i]记录i的最小质因子的次数
int d[N]; //d[i]记录i的约数个数

void get_d(int n){ //筛法求约数个数
    d[1] = 1;
    for(int i=2; i<=n; i++){
        if(!vis[i]){
            p[++cnt] = i;
            a[i] = 1; d[i] = 2;
        }
        for(int j=1; i*p[j]<=n; j++){
            int m = i*p[j];
            vis[m] = 1;
            if(i%p[j] == 0){
                a[m] = a[i]+1;
                d[m] = d[i]/a[m]*(a[m]+1);
                break;
            }
            else{
                a[m] = 1; d[m] = d[i]*2;
            }
        }
    }
}
```

G11 筛法求约数和

问题:

给一个数 n ($n \leq 1 \times 10^6$), 输出 $1 \sim n$ 每个数的约数和。

约数和定理

若 $n = \prod_{i=1}^s p_i^{\alpha_i}$, 则 $f(n) = \prod_{i=1}^s \sum_{j=0}^{\alpha_i} p_i^j$

证明: $p_i^{\alpha_i}$ 的约数有 $p_i^0, p_i^1, \dots, p_i^{\alpha_i}$ 共 $(\alpha_i + 1)$ 个,

其约数和为 $\sum_{j=0}^{\alpha_i} p_i^j$,

根据乘法原理, $f(n) = \prod_{i=1}^s \sum_{j=0}^{\alpha_i} p_i^j$

例: $12 = 2^2 \times 3^1$,

$$f(12) = \{1 + 2 + 4\} \times \{1 + 3\} = 7 \times 4 = 28$$

筛法求约数和

若 i 是质数, $g[i] = f[i] = i + 1$ 。

在线性筛中, 每个合数 m 都是被最小的质因子筛掉的。

设 p_j 是 m 的最小质因子, 则 m 通过 $m = i \times p_j$ 筛掉。

(1) 若 i 能被 p_j 整除, 则 p_j 一定也是 i 的最小质因子

$$g[i] = p_j^0 + p_j^1 + \dots + p_j^{\alpha_j}, \quad g[m] = p_j^0 + p_j^1 + \dots + p_j^{\alpha_j+1}$$

$$f[i] = g[i] \times \dots, \quad f[m] = g[m] \times \dots$$

(2) 若 i 不能被 p_j 整除, 则 i 不包含质因子 p_j 。

$$g[m] = 1 + p_j$$

$$f[m] = g[m] \times f[i]$$

```
const int N = 1000010;
int p[N], vis[N], cnt;
//g[i]表示i的最小质因子的1+p^1+...+p^k
int g[N], f[N]; //f[i]表示i的约数和

void get_f(int n){ //筛法求约数和
    g[1] = f[1] = 1;
    for(int i=2; i<=n; i++){
        if(!vis[i]){
            p[++cnt] = i;
            g[i] = f[i] = i+1;
        }
        for(int j=1; i*p[j]<=n; j++){
            int m = i*p[j];
            vis[m] = 1;
            if(i%p[j] == 0){
                g[m] = g[i]*p[j]+1;
                f[m] = f[i]/g[i]*g[m];
                break;
            }
            else{
                g[m] = p[j]+1;
                f[m] = f[i]*g[m];
            }
        }
    }
}
```

G13 费马小定理

没啥好说的, 直接记结论得了。

问题

给两个数 a, p , p 是质数, 求 a 模 p 的乘法逆元

由费马小定理

$a^{p-1} \equiv 1 \pmod{p}$, 得 $a \times a^{p-2} \equiv 1 \pmod{p}$,

所以 a^{p-2} 就是 a 模 p 的乘法逆元。

用快速幂求即可。

G14 拓展欧拉定理-超大幂次取余

没啥好说的, 这个只需要记住公式就行了。

Luogu P5091 【模板】扩展欧拉定理

给你三个正整数 a, m, b , 你需要求: $a^b \bmod m$

数据范围

$$1 \leq a \leq 10^9, \quad 1 \leq m \leq 10^8, \quad 1 \leq b \leq 10^{200000000}$$

欧拉定理

若 $\gcd(a, m) = 1$, 则 $a^{\varphi(m)} \equiv 1 \pmod{m}$

扩展欧拉定理

$$a^b \equiv \begin{cases} a^b, & b < \varphi(m), \\ a^{b \bmod \varphi(m) + \varphi(m)}, & b \geq \varphi(m), \end{cases} \pmod{m}$$

当 b 小的时候, 不用降幂, 直接跑快速幂即可。

当 b 是大幂幂的时候, 先降成小幂幂, 再跑快速幂。

$$\text{例: } a = 2, m = 4, b = 1, 2 \not\equiv 2^{1 \bmod 2 + 2} \pmod{4}$$

$$\text{例: } a = 2, m = 4, b = 6, 2 \equiv 2^{6 \bmod 2 + 2} \pmod{4}$$

```
int get_phi(int m){ //求欧拉函数
    int res = m;
    for(int i=2; i*i<=m; i++){
        if(m%i == 0){
            res = res/i*(i-1);
            while(m%i == 0) m /= i;
        }
    }
    if(m>1) res = res/m*(m-1);
    return res;
}

int depow(int phi){ //降幂
    int b = 0;
    for(int i=0; s[i]; i++){
        b = b*10+(s[i]-'0');
        if(b>=phi) flag=1, b%=phi;
    }
    if(flag) b += phi;
    return b;
}
```

G17 拓展欧几里得-不定方程

问题：求 $ax + by = \gcd(a, b)$ 的一组整数解

扩展欧几里得算法

当 $b = 0$ 时 $ax + by = a$ 故而 $x = 1, y = 0$

当 $b \neq 0$ 时

由欧几里得算法, $\gcd(a, b) = \gcd(b, a \% b)$

由裴蜀定理,

$$\gcd(a, b) = ax + by$$

$$\gcd(b, a \% b) = bx_1 + (a \% b)y_1$$

$$= bx_1 + (a - \left\lfloor \frac{a}{b} \right\rfloor \times b)y_1$$

$$= ay_1 + b(x_1 - \frac{a}{b}y_1)$$

$$\text{所以 } x = y_1, y = x_1 - \frac{a}{b}y_1$$

可以用递归算法, 先求出下一层的 x_1, y_1 ,

再回代到上一层, 层层回代, 可求特解 (x_0, y_0)

构造通解

$$\begin{cases} x = x_0 + \frac{b}{\gcd(a, b)} * k \\ y = y_0 - \frac{a}{\gcd(a, b)} * k \end{cases} \quad (\text{考虑 } ax + by = 0 \text{ 构造})$$

例: $8x + 6y = 2$, 解: $(1, -1), (4, -5), (7, -9) \dots$

```
int gcd(int a, int b){
    if(b == 0) return a;
    return gcd(b, a % b);
}
```

```
int exgcd(int a, int b, int &x, int &y){
    if(b == 0){
        x = 1, y = 0;
        return a;
    }
    int x1, y1, d;
    d = exgcd(b, a % b, x1, y1);
    x = y1, y = x1 - a / b * y1;
    return d;
}
```

模拟:

```
exgcd(8, 6, x, y) { d=exgcd(6, 2, x, y), ... }
exgcd(6, 2, x, y) { d=exgcd(2, 0, x, y), ... }
exgcd(2, 0, x, y) { x=1, y=0, return 2 }
(6, 2, x, y) { d=2, x=0, y=1-6/2*0=1, return 2 }
(8, 6, x, y) { d=2, x=1, y=0-8/6*1=-1, return 2 }
```

时间复杂度: $O(\log n)$

G18 拓展欧几里得-乘法逆元

问题

给定整数 a, b, m , 求解同余方程 $ax \equiv b \pmod{m}$,

如果 x 存在整数解, 则输出任意一个;

如果不存在, 则输出 none

例: $8x \equiv 4 \pmod{6}$, 整数解 $x = 2$

扩欧算法求解线性同余方程

1. 把同余方程转化为不定方程

由 $ax \equiv b \pmod{m}$

得 $ax = m(-y) + b$

即 $ax + my = b$

由裴蜀定理, 当 $\gcd(a, m) | b$ 时有解

2. 用扩欧算法求 $ax + my = \gcd(a, m)$ 的解

把 x 乘以 $b/\gcd(a, m)$ 即得原方程的特解

```
int exgcd(int a, int b, int &x, int &y){
    if(b == 0){
        x = 1, y = 0; return a;
    }
    int x1, y1, d;
    d = exgcd(b, a % b, x1, y1);
    x = y1, y = x1 - a / b * y1;
    return d;
}
```

```
int main(){
    int a, b, m, x, y;
    scanf("%d%d%d", &a, &b, &m);
    int d = exgcd(a, m, x, y);
    if(b % d == 0)
        printf("%d", 111 * x * b / d % m);
}
```

时间复杂度: $O(\log n)$

G20 扩展中国剩余定理

问题

求解线性同余方程组

$$\begin{cases} x \equiv r_1 \pmod{m_1} \\ x \equiv r_2 \pmod{m_2} \\ \dots \\ x \equiv r_n \pmod{m_n} \end{cases}$$

其中 $m_1, m_2, \dots, m_n$ 为不一定两两互质的整数，求 x 的最小非负整数解。

中国剩余定理 (CRT) 已不可行

其构造解 $x = \sum_{i=1}^n r_i c_i c_i^{-1} \pmod{M}$

其中 $c_i x \equiv 1 \pmod{m_i}$ ，即 $c_i x + m_i y = 1 = \gcd(c_i, m_i)$

根据裴蜀定理， c_i, m_i 应该互质， $c_i = \frac{m_1 \times \dots \times m_n}{m_i}$

如果 c_i, m_i 不互质，则 c_i^{-1} 不存在，算法失效

扩展中国剩余定理 (EXCRT)

前两个方程： $x \equiv r_1 \pmod{m_1}, x \equiv r_2 \pmod{m_2}$

转化为不定方程： $x = m_1 p + r_1 = m_2 q + r_2$

则 $m_1 p - m_2 q = r_2 - r_1$

由裴蜀定理，

当 $\gcd(m_1, m_2) \nmid (r_2 - r_1)$ 时，无解

当 $\gcd(m_1, m_2) \mid (r_2 - r_1)$ 时，有解

由扩欧算法，

得特解 $p = p * \frac{r_2 - r_1}{\gcd}, q = q * \frac{r_2 - r_1}{\gcd}$

其通解 $P = p + \frac{m_2}{\gcd} * k, Q = q - \frac{m_1}{\gcd} * k$

所以 $x = m_1 P + r_1 = \frac{m_1 m_2}{\gcd} * k + m_1 p + r_1$

前两个方程等价合并为一个方程 $x \equiv r \pmod{m}$

其中 $r = m_1 p + r_1, m = \text{lcm}(m_1, m_2)$

所以 n 个同余方程只要合并 $n - 1$ 次，即可求解

时间复杂度： $O(n \log(m_1))$

G22 扩展BSGS算法

求解高次同余方程

给定整数 a, b, p ,

求满足 $a^x \equiv b \pmod{p}$ 的最小非负整数 x 。

时间复杂度： $O(\sqrt{p})$

G23 高斯消元法

题目描述

给定一个线性方程组，对其求解。

$$\begin{cases} a_{1,1}x_1 + a_{1,2}x_2 + \dots + a_{1,n}x_n = b_1 \\ a_{2,1}x_1 + a_{2,2}x_2 + \dots + a_{2,n}x_n = b_2 \\ \dots \\ a_{n,1}x_1 + a_{n,2}x_2 + \dots + a_{n,n}x_n = b_n \end{cases}$$

输入格式

第一行，一个正整数 n 。

第二至 $n + 1$ 行，每行 $n + 1$ 个整数，为 $a_1, a_2, \dots, a_n$ 和 b ，代表一组方程。

输出格式

共 n 行，每行一个数，第 i 行为 x_i （四舍五入保留 2 位小数）。

如果不存在唯一解或无解，在第一行输出 No Solution。

输入

```
3
1 3 4 5
1 4 7 3
9 3 2 2
```

输出

-0.97
5.18
-2.39

说明/提示

数据范围: $1 \leq n \leq 100, |a_i| \leq 10^4, |b| \leq 10^4$ 。保证数据若有解则所有解均满足 $|x_i| \leq 10^3$, 且 $x_i \pm 10^{-6}$ 和 x_i 四舍五入后的结果相同 (即不会因为较小的精度误差导致四舍五入后的结果不同)。

G27 求组合数-卢卡斯

大组合数取模问题

给定整数 n, m, p 的值, 求出 $C_n^m \pmod p$ 的值。

其中 $1 \leq m \leq n \leq 10^{18}, 1 \leq p \leq 10^5$, 保证 p 为质数。

时间复杂度: $O(p \log p + \log_p n)$

G29 隔板法

不是哥们, 这还需要板子?

给一下吧哈哈

隔板法 (Stars and Bars)

1. 求线性不定方程的整数解的组数
2. 求相同元素分组的方案数

1. 正整数和的组数

若 $x_i \geq 1$, 求 $x_1 + x_2 + \cdots + x_k = n$ 的整数解的组数。

现有 n 个完全相同的元素, 将其分为 k 组, 保证每组至少有一个元素, 一共有多少种分法?

把 n 个相同的球排成一行, 有 $n-1$ 个空。

拿 $k-1$ 块板子插入到 $n-1$ 个空里, 把球分成 k 组。
即在 $n-1$ 个空里选择 $k-1$ 个空插板子, 所以答案就是 C_{n-1}^{k-1}

例 $n=4, k=3$ $C_{4-1}^{3-1} = C_3^2 = 3$

(1, 1, 2) (1, 2, 1) (2, 1, 1)
○ | ○ | ○ ○ ○ | ○ ○ | ○ ○ ○ | ○ | ○

2. 非负整数和的组数

若 $x_i \geq 0$, 求 $x_1 + x_2 + \cdots + x_k = n$ 的整数解的组数。

令 $y_i = x_i + 1$, 则 $y_i \geq 1$

则 $y_1 + y_2 + \cdots + y_k = n + k = m$

转化为情况1, 所以答案是 $C_{m-1}^{k-1} = C_{n+k-1}^{k-1}$

3. 不同下界整数和的组数

求 $x_1 + x_2 + \cdots + x_k = n$ 的整数解的组数,

其中 $x_i \geq a_i \geq 0, \sum a_i \leq n$ 。

令 $y_i = x_i - a_i + 1$, 则 $y_i \geq 1$

则 $y_1 + y_2 + \cdots + y_k = n - \sum a_i + k = m$

转化为情况1, 所以答案为 $C_{m-1}^{k-1} = C_{n-\sum a_i+k-1}^{k-1}$

G32 卡特兰数

Catalan 通项公式

$$(1) H_n = C_{2n}^n - C_{2n}^{n-1} \quad (2) H_n = \frac{1}{n+1} C_{2n}^n \quad (3) H_n = \frac{4n-2}{n+1} H_{n-1}$$

Catalan 数列

H_0	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8
1	1	2	5	14	42	132	429	1430

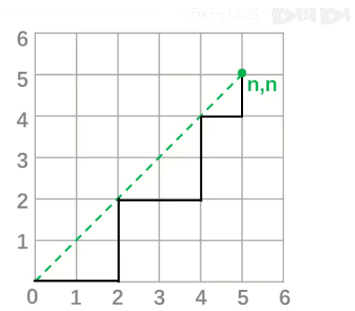
Catalan 特征

从 $(0,0)$ 到 (n,n) ，不越过对角线，即任何时候，向上走的步数不能超过向右走的步数。

一种操作数不能超过另外一种操作数，或者两种操作不能有交集，这些操作的合法方案数，通常是卡特兰数。

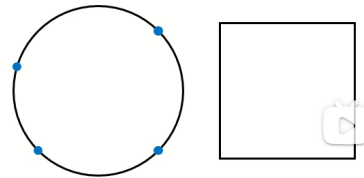
Catalan 应用

1. 一个有 n 个 0 和 n 个 1 组成的字串，且所有的前缀字串皆满足 1 的个数不超过 0 的个数。这样的字串个数有多少？
2. 包含 n 组括号的合法运算式的个数有多少？
3. 一个栈的进栈序列为 $1, 2, 3, \dots, n$ ，有多少个不同的出栈序列？
4. n 个结点可构造多少个不同的二叉树？
5. 在圆上选择 $2n$ 个点，将这些点成对连接起来使得所得到的 n 条弦不相交的方法数？
6. 通过连结顶点而将 $n+2$ 边的凸多边形分成 n 个三角形的方法数？



合法：0011,0101

非法：0110,1001,1010,1100



G33 整除分块

整除分块

$$\text{求} \sum_{i=1}^n \left\lfloor \frac{n}{i} \right\rfloor$$

例： $n = 21$ 时， $\left\lfloor \frac{n}{i} \right\rfloor$ 的值分布在 8 个块内

i	1	2	3	4	5	6	7	8	9	10	11	...	21
$\left\lfloor \frac{n}{i} \right\rfloor$	21	10	7	5	4	3	2	2	2	2	1	...	1

性质1：分块的块数 $\leq 2\sqrt{n}$

当 $i \leq \sqrt{n}$ 时， $\left\lfloor \frac{n}{i} \right\rfloor$ 有 $\sqrt{n}$ 种取值。

当 $i > \sqrt{n}$ 时， $\left\lfloor \frac{n}{i} \right\rfloor \leq \sqrt{n}$ ， $\left\lfloor \frac{n}{i} \right\rfloor$ 至多有 $\sqrt{n}$ 种取值。

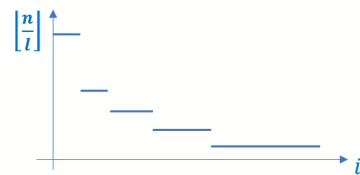
性质2： i 所在块的右端点为 $\left\lfloor \frac{n}{\left\lfloor \frac{n}{i} \right\rfloor} \right\rfloor$

i 所在块的值 $k = \left\lfloor \frac{n}{i} \right\rfloor$ ，则 $k \leq \frac{n}{i}$ ，则 $\left\lfloor \frac{n}{k} \right\rfloor \geq \left\lfloor \frac{n}{\frac{n}{i}} \right\rfloor = \left\lfloor i \right\rfloor = i$

所以 $i_{\max} = \left\lfloor \frac{n}{k} \right\rfloor = \left\lfloor \frac{n}{\left\lfloor \frac{n}{i} \right\rfloor} \right\rfloor$ 。代码实现时，右端点 $r = n/(n/l)$;

$$\text{求} \sum_{i=1}^n f(i) \left\lfloor \frac{n}{i} \right\rfloor$$

先预处理出 $f(i)$ 的前缀和 $s(i) = \sum_{j=1}^i f(j)$ ，再枚举每一块 $[l, r]$ ，累加每块的贡献。



```
for(int l = 1; l <= n; l = r + 1){
    r = n/(n/l);
    res += (s(r) - s(l - 1)) * (n/l);
}
```

时间复杂度： $O(\sqrt{n})$

Luogu P2261 [CQOI2007]余数求和

给出正整数 n 和 k ，请计算 $G(n, k) = \sum_{i=1}^n k \bmod i$ ，其中 $k \bmod i$ 表示 k 除以 i 的余数。

对于 100% 的数据，保证 $1 \leq n, k \leq 10^9$ 。

思路

$$\begin{aligned} \text{res} &= \sum_{i=1}^n k \% i \\ &= \sum_{i=1}^n (k - i * \left\lfloor \frac{k}{i} \right\rfloor) \\ &= n * k - \sum_{i=1}^n i * \left\lfloor \frac{k}{i} \right\rfloor \end{aligned}$$

后一项对于每个区块 $[l, r]$ 都是等差数列

$$\begin{aligned} \sum_{i=l}^r i * \left\lfloor \frac{k}{i} \right\rfloor &= (k/l) * \sum_{i=l}^r i \\ &= (k/l) * (r - l + 1) * (l + r) / 2 \\ &= (k/l) * \text{区间宽度} * \text{区间平均值} \end{aligned}$$

```
int main(){
    LL n, k, l, r, res;
    scanf("%lld%lld", &n, &k);
    res = n*k;
    for(l=1; l<=n; l=r+1){
        if(k/l == 0) break;
        r = min(k/(k/l), n);
        res -= (k/l)*(r-l+1)*(l+r)/2;
    }
    printf("%lld", res);
    return 0;
}
```

G41 FFT-多项式乘法

题目描述

给定一个 n 次多项式 $F(x)$ ，和一个 m 次多项式 $G(x)$ 。

请求出 $F(x)$ 和 $G(x)$ 的乘积。

输入格式

第一行两个整数 n, m 。

接下来一行 $n + 1$ 个数字，从低到高表示 $F(x)$ 的系数。

接下来一行 $m + 1$ 个数字，从低到高表示 $G(x)$ 的系数。

输出格式

一行 $n + m + 1$ 个数字，从低到高表示 $F(x) \cdot G(x)$ 的系数。

输入 #1

```
1 2
1 2
1 2 1
```

输出 #1

```
1 4 5 2
```

说明/提示

保证输入中的系数大于等于 0 且小于等于 9。

对于 100% 的数据: $1 \leq n, m \leq 10^6$ 。

有没有讲解？没有，会用就行了！！！！

G45+G46 第一、二类斯特林数

不讲解，打表找到规律后直接使用！

第一类斯特林数（斯特林轮换数）

将 n 个不同元素，划分为 m 个非空圆排列的方案数，记做 $S(n, m)$ 或 $\left[\begin{smallmatrix} n \\ m \end{smallmatrix} \right]$

$$\text{递推式} \quad \left[\begin{smallmatrix} n \\ m \end{smallmatrix} \right] = \left[\begin{smallmatrix} n-1 \\ m-1 \end{smallmatrix} \right] + (n-1) \left[\begin{smallmatrix} n-1 \\ m \end{smallmatrix} \right]$$

$$\text{边界} \quad S[i][0] = [i = 0]$$

$[n \setminus m]$	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	2	3	1			
4	0	6	11	6	1		
5	0	24	50	35	10	1	
6	0	120	274	225	85	15	1

第二类斯特林数（斯特林子集数）

将 n 个不同元素，划分为 m 个非空子集的方案数，记做 $S(n, m)$ 或 $\left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\}$

$$\text{递推式} \quad \left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} n-1 \\ m-1 \end{smallmatrix} \right\} + m \left\{ \begin{smallmatrix} n-1 \\ m \end{smallmatrix} \right\}$$

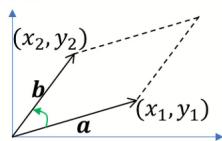
$$\text{边界} \quad S[i][0] = [i = 0]$$

$\{n \setminus m\}$	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	1	3	1			
4	0	1	7	6	1		
5	0	1	15	25	10	1	
6	0	1	31	90	65	15	1

G49 向量运算

叉积

- 定义: $\mathbf{a} \times \mathbf{b} = |\mathbf{a}||\mathbf{b}|\sin\theta$, 例如 力矩 $M = RF\sin\theta$
- 几何意义: 向量 $\mathbf{a}$ 与 $\mathbf{b}$ 张成的平行四边形的有向面积
 $\mathbf{b}$ 在 $\mathbf{a}$ 的逆时针方向为正, 顺时针方向为负
- 坐标运算: $\mathbf{a} \times \mathbf{b} = x_1y_2 - y_1x_2$



证明:

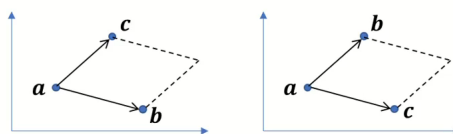
$$\begin{aligned}\mathbf{a} \cdot \mathbf{b} &= |\mathbf{a}||\mathbf{b}|\cos\theta = x_1x_2 + y_1y_2 \\ (\mathbf{a} \times \mathbf{b})^2 + (\mathbf{a} \cdot \mathbf{b})^2 &= a^2b^2 \\ (\mathbf{a} \times \mathbf{b})^2 &= a^2b^2 - (\mathbf{a} \cdot \mathbf{b})^2 \\ &= (x_1^2 + y_1^2)(x_2^2 + y_2^2) - (x_1x_2 + y_1y_2)^2 \\ &= (x_1y_2 - y_1x_2)^2\end{aligned}$$

代码实现

```
double cross(Point a, Point b){
    return a.x*b.y-a.y*b.x;
}
```

4. 应用: 判定点线的位置关系

- $$\begin{aligned}(\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a}) &> 0 \Rightarrow \mathbf{c} \text{ 点在直线 } \mathbf{ab} \text{ 的左侧} \\ (\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a}) &< 0 \Rightarrow \mathbf{c} \text{ 点在直线 } \mathbf{ab} \text{ 的右侧} \\ (\mathbf{b}-\mathbf{a}) \times (\mathbf{c}-\mathbf{a}) &= 0 \Rightarrow \text{三点共线}\end{aligned}$$



```
Point operator-(Point a, Point b){
    return Point(a.x-b.x, a.y-b.y);
}
double operator*(Point a, Point b){
    return a.x*b.y-a.y*b.x;
}
double cross(Point a, Point b, Point c){
    return (b-a)*(c-a);
}
```

```
double cross(Point a, Point b, Point c){
    return (b.x-a.x)*(c.y-a.y)-(b.y-a.y)*(c.x-a.x);
}
```

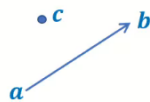
G50 线线关系

叉积 $\mathbf{a} \times \mathbf{b} = |\mathbf{a}||\mathbf{b}|\sin\theta$

$$\mathbf{a} \times \mathbf{b} = x_1y_2 - y_1x_2$$

1. 判定点线的位置关系

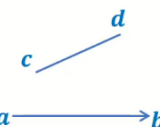
- $\mathbf{c}$ 点在直线 $\mathbf{ab}$ 的左侧: $\text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{c}) > 0$
 $\mathbf{c}$ 点在直线 $\mathbf{ab}$ 的右侧: $\text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{c}) < 0$
 三点共线: $\text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{c}) = 0$



2. 判定线线的位置关系

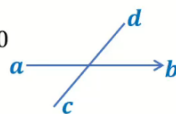
(1) 直线 $\mathbf{ab}$ 与线段 $\mathbf{cd}$ 无交点:

$$\text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{c}) * \text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{d}) > 0$$



(2) 直线 $\mathbf{ab}$ 与线段 $\mathbf{cd}$ 有交点:

$$\text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{c}) * \text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{d}) \leq 0$$



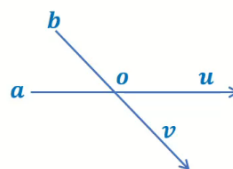
(3) 线段 $\mathbf{ab}$ 与线段 $\mathbf{cd}$ 无交点

$$\begin{aligned}\text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{c}) * \text{cross}(\mathbf{a}, \mathbf{b}, \mathbf{d}) &> 0 \text{ 或} \\ \text{cross}(\mathbf{c}, \mathbf{d}, \mathbf{a}) * \text{cross}(\mathbf{c}, \mathbf{d}, \mathbf{b}) &> 0\end{aligned}$$

(4) 求两直线的交点

两点式 $(\mathbf{a}, \mathbf{c}) \rightarrow$ 点向式 $(\mathbf{a}, \mathbf{c}-\mathbf{a})$

```
Point getNode(Point a, Point u, Point b, Point v){
    double t=(a-b)*v/(v*u);
    return a+u*t;
}
```



G52 凸包算法

没有py代码, 因为EH还不会。

题目描述

农夫约翰想要建造一个围栏用来围住他的奶牛, 可是他资金匮乏。他建造的围栏必须包括他的奶牛喜欢吃草的所有地点。对于给出的这些地点的坐标, 计算最短的能够围住这些点的围栏的长度。

输入格式

输入数据的第一行是一个整数。表示农夫约翰想要围住的放牧点的数目 n 。

第 2 到第 $(n+1)$ 行, 每行两个实数, 第 $(i+1)$ 行的实数 x_i, y_i 分别代表第 i 个放牧点的横纵坐标。

输出格式

输出输出一行一个四舍五入保留两位小数的实数，代表围栏的长度。

输入输出样例 #1

输入 #1

```
4
4 8
4 12
5 9.3
7 8
```

输出 #1

```
12.00
```

对于 100% 的数据，保证 $1 \leq n \leq 10^5$ ， $-10^6 \leq x_i, y_i \leq 10^6$ 。小数点后最多有 2 位数字。

G53 旋转卡壳

题目描述

给定平面上 n 个点，求凸包直径。

输入格式

第一行一个正整数 n 。

接下来 n 行，每行两个整数 x, y ，表示一个点的坐标。保证所有点的坐标两两不同。

输出格式

输出一行一个整数，表示答案的平方。

输入输出样例 #1

输入 #1

```
4
0 0
0 1
1 1
1 0
```

输出 #1

```
2
```

对于 100% 的数据， $2 \leq n \leq 50000$ ， $|x|, |y| \leq 10^4$ 。

G99 超级gcd

Problem J. 象牙

Input file: standard input
Output file: standard output
Time limit: 5 seconds
Memory limit: 1024 megabytes

给定四个正整数 a, b, c, d 。计算 $\gcd(a^b, c^d)$ 的值，并将其对 998244353 取模。

Input

第一行包含一个整数 t ($1 \leq t \leq 10^5$)，表示测试用例的数量。
对于每个测试用例，输入只有一行，包含四个正整数 a, b, c, d ($1 \leq a, b, c, d \leq 10^{18}$)。

Output

对于每个测试用例，输出一行一个整数，表示 $\gcd(a^b, c^d)$ 对 998244353 取模后的结果。

Example

standard input	standard output
5	1
2 3 3 2	8
4 2 8 1	9
6 2 9 1	1
7 1 11 1	586315999
1000000000000000 1 1000000000000000 1	

H 邪教

H1 哈希状态

Problem Description

在一个无限长的数轴上，有 n 个集合，每个集合给定一个范围 $[l, r]$ ，其中 l 和 r 为整数，且满足 $l \leq r$ 。我们称两个整数 x 和 y 性质不同，当且仅当存在至少一个集合 S ，使得 x 属于 S 但 y 不属于 S ，或者 x 不属于 S 但 y 属于 S 。

你的任务是计算在这个数轴上最多可以选出多少个数，使得这些数的性质两两不同。

Input

第一行包含一个整数 t ，表示测试样例的组数 ($1 \leq t \leq 10000$)。

每组测试样例的格式如下：

- 第一行包含一个整数 n ，表示集合的数量 ($0 \leq n \leq 200000$)。
- 接下来的 n 行，每行包含两个整数 l 和 r ，表示集合的范围 ($0 \leq l \leq r \leq 10^9$)。

保证所有测试样例的 n 的总和不超过 10^6 。

Output

对于每组测试样例，输出一个整数，表示数轴上最多可以选出的性质两两不同的数的数量。

Sample Input

```
3
1
1 6
4
0 12
4 13
6 13
12 13
0
```

Sample Output

```
2
6
1
```

解析：时间复杂度 $O(n \log n)$)

想到端点以及单调栈算法后，剩下的就是状态维护问题了，每次进栈出栈，维护不同时候的集合的不同状态数量，可以使用哈希异或的方法，WA的概率非常小。

H2 超绝浮点精度

本邪教为py邪教，方法是使用python的Decimal提升精度，在WA与T之间徘徊。

题目描述

“荒泷卡牌天下第一斗”酷爱卡牌，尤其是最近马上要上线的“七圣召唤”，荒泷一斗想在2.3版本前集齐全部种类的卡牌，但他不精于计算，所以他找来了你、战力第一的旅行者帮忙。

已知“七圣召唤”共有K张不同的卡牌，荒泷一斗的资金够买N个卡包。每个卡包能随机从K种卡牌中抽到一张。现给出N和K，他想问你要集齐全部种类的卡牌所需的期望卡包数，以及N个卡包能开出的期望卡牌种类数。

输入描述：

第一行输入两个整数，N和K。N表示卡包数，K表示卡牌种类数。（ $1 \leq N \leq 1000000, 1 \leq K \leq 1000000$ ）

输出描述：

第一行输出两个浮点数，分别是集齐全部种类的卡牌所需的期望卡包数，和N个卡包能开出的期望卡牌种类数，中间用空格间隔。（答案与标准答案相差 $1e-6$ 以内就算正确）

输入1

3 3

输出1

5.50000000 2.11111111

输入2

4 3

输出2

5.50000000 2.40740741